



**School Of Engineering
Electrical And Computer Engineering**

Date: June 13, 2026

I HEREBY RECOMMEND THAT THE THESIS PREPARED UNDER MY
SUPERVISION BY

Luca Chierotti

ENTITLED

**Automated Freestyle Stroke Count and Phase Detection via
Synthetic Video Data for Competitive Swimming
Applications**

BE ACCEPTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR
THE DEGREE

OF

**MASTER OF SCIENCE IN ELECTRICAL AND COMPUTER
ENGINEERING**

MKyrarini

Thesis Advisor
Dr. Maria Kyrarini

Sally Wood

Thesis Reader
Dr. Sally Wood

Chair of Department
Dr. Shoba Krishnan

**Automated Freestyle Stroke Count and Phase Detection via
Synthetic Video Data for Competitive Swimming
Applications**

By

Luca Chierotti

Graduate Master's Thesis

Submitted in Partial Fulfillment of the Requirements
for the Degree of Master Of Science
in the School of Engineering at
Santa Clara University, June 2026

Santa Clara, California

Acknowledgements

I would like to thank my thesis and academic advisor, Dr. Maria Kyrarini, for her support and guidance during the completion of my master's thesis. Her advice and expertise played a significant role throughout this project and my 6 years at Santa Clara. Additionally, I would like to thank Dr. Sally Wood for being a thesis reader. Furthermore, I want to acknowledge the use of AI tools such as Grammarly with helping write and edit this thesis. Finally, I am grateful to all my friends and family for their support during my master's thesis and studies at university.

Abstract

Competitive swimming performance depends on small differences in stroke timing and body motion, but detailed video analysis is difficult to perform consistently at scale because manual review is slow, subjective, and labor-intensive. Automated video-based analysis could make stroke evaluation more repeatable and accessible, but swimming remains challenging because water, splashes, reflections, bubbles, occlusion, and partial submersion can make joint pose estimation for swimmers difficult.

This thesis develops and evaluates a video-based pipeline for freestyle stroke-phase classification and stroke counting using the SwimXYZ synthetic swimming dataset. Using this dataset, a pipeline is developed to automatically generate stroke-phase labels, enabling training without manually labeled real-world data. Freestyle arm motion is modeled using four phases: Recovery, Entry/Catch, Pull, and Push. The contributions of this thesis are: (1) a synthetic swimming dataset pipeline that automatically generates Common Objects in Context (COCO)-17-style pose keypoints and frame-level freestyle phase labels; (2) a task-specific comparison of YOLO and MediaPipe pose-estimation models using localization error, missing-keypoint rate, jitter, wrist-weighted performance, and a composite downstream score; (3) a YOLO extraction pipeline that converts videos into a windowed dataset; (4) a dual-arm Long Short-Term Memory (LSTM) model for frame-level left- and right-arm phase classification; and (5) a stroke-counting method using Hidden Markov Model (HMM)/Viterbi smoothing and rule-based phase-cycle counting. The final pipeline uses YOLO applied to videos from the side-above-water viewpoint, LSTM phase classification, HMM/Viterbi smoothing, and rule-based phase-cycle counting to estimate completed freestyle strokes. Using perfect (ground truth) COCO-17 pose inputs, the LSTM achieved 0.9469 test accuracy, establishing that the temporal model can learn freestyle phase structure when pose information is accurate. Using YOLO-derived side-above-water keypoints, the LSTM achieved 0.8363 test accuracy, showing that performance decreases with noisy and missing computer-vision keypoints but remains meaningful for phase classification. In a 20-clip manual validation set, the final stroke-counting pipeline achieved a total mean absolute error of 0.80 strokes and was within one stroke of the manual count in 85.00% of clips. These results show that synthetic pose and phase labels can support an interpretable freestyle swimming analysis pipeline. The system still produces meaningful phase predictions and stroke counts, providing a foundation for future real-world markerless swim analysis.

Contents

Acknowledgements	i
Abstract	iii
1 Introduction	9
1.1 Background and Motivation	9
1.2 Problem Statement	12
1.3 Thesis Contributions	13
2 Related Work	15
2.1 Pose Estimation in Aquatic Environments	15
2.2 Video-Based Stroke Detection	17
2.3 Inertial Sensors for Stroke Phase Detection	20
2.4 Research Gap	22
3 SwimXYZ Dataset and Automatic Phase Labeling	25
3.1 SwimXYZ Dataset	25
3.2 Automatic Phase Labeling	32
4 Proposed Framework for Freestyle Phase Classification and Stroke Counting	41
4.1 System Overview	41

4.2	Computer Vision Pose Extraction	42
4.3	Pose Normalization and Windowing	47
4.4	LSTM Phase Classification	50
4.5	Hidden Markov Model and Viterbi Temporal Decoding	54
4.6	Stroke Counting	57
5	Results & Experiments	59
5.1	LSTM Performance Using Perfect COCO Pose Inputs	59
5.2	MediaPipe and YOLO Pose-Estimation Comparison	63
5.3	YOLO Side-Above-Water Dataset and LSTM Results	66
5.3.1	Developing the Side-Above-Water View Dataset	66
5.3.2	Side-Above-Water View LSTM Model Results	70
5.3.3	Manual Stroke Count Validation	73
6	Conclusion	77

List of Figures

1.1	Arm stroke phases in front-crawl (classification proposed by Chollet et al., 2000) and graphical presentation of reference system axes [1]. . .	10
3.1	Sample images from SwimXYZ for freestyle motions from the various viewpoints. The top left, middle, and right are the Aerial, Front, and Side Above Water views respectively. The bottom left and right are the Side Underwater and Side Water Level Right views respectively. .	26
3.2	Structure of the possible hyperparameter settings for each video . . .	27
3.3	SwimXYZ pipeline for generating swimming videos	28
3.4	Side Water view of swimmer with overlaid x,y,z coordinate system used in SwimXYZ dataset	30
3.5	Cross sections of the swimmer's right arm. The grey ellipse represents the swimmer's head. Left: view from behind swimmer. Middle: top view of swimmer. Right: right side view of swimmer	30
3.6	The Base data annotations over a full 5 second video	31
3.7	The cyclic motions of relevant joints in the ZY plane over a 5 second sample video from the Base annotation set	31
3.8	The before and after of the data preprocessing on the base data for a sample video. The data already normalized by the hip joint. The absolute value of the Z-coordinate is taken.	33

3.9	The before and after of the data preprocessing on the COCO data for a sample video. The data is normalized by the hip joint and the absolute value of the X-coordinate is taken.	34
3.10	Shoulder flexion angle compared to the Sagital and Scapular Planes [2].	35
3.11	The right hand motions over a 5 second sample video from the Base annotation set. The left plot displays the right hand X, Y, and Z motions while the right plot displays the cyclic hand motion in the ZY plane for the same video	37
3.12	How the shoulder elevation angle of a swimmer corresponds to the freestyle stroke phase. This is a side view perspective of the swimmer (ZY plane), meaning theoretical swimmer's head rests at 0 degrees while the feet lay at 180 degrees relative to the swimmers right shoulder joint.	37
3.13	The four calculated freestyle swim phase classes overlaid onto the left and right hand positions and shoulder elevation angles for a sample 5 second SwimXYZ video	38
4.1	Overview of the Automatic Freestyle Stroke Counting Architecture. .	42
4.2	Overview of the Automatic Freestyle Stroke Counting Architecture. .	49
4.3	Architecture of the dual-arm LSTM model used for frame-level freestyle phase classification. The model receives 60-frame pose windows with 18 pose features per frame and outputs phase logits for the left and right arms at each frame.	52
5.1	Normalized confusion matrices for the COCO LSTM model. Values represent the proportion of each true class assigned to each predicted class.	61

5.2	Example video frames showing the harsh aquatic environment, including reflected sunlight and bubbles, making CV inference difficult. . . .	67
5.3	YOLO rectification example for the left wrist in the side-above-water view. The plot shows raw image coordinates, MidHip-normalized coordinates, rectified coordinates, and the wrist presence mask.	68
5.4	YOLO rectification example for the right wrist in the side-above-water view. The presence mask shows frames where YOLO detected or missed the wrist keypoint.	69
5.5	Normalized confusion matrices for the YOLO side-above-water LSTM model. Pull was the most difficult phase, with many Pull frames confused with Entry/Catch or Push.	72

List of Tables

4.1	Summary of pose-estimation evaluation metrics.	46
5.1	Summary of COCO LSTM training and test performance.	60
5.2	Combined left- and right-arm classification report for the COCO LSTM model.	62
5.3	COCO LSTM test accuracy by camera view.	62
5.4	COCO LSTM test accuracy by swimmer body appearance.	62
5.5	COCO LSTM test accuracy by swimming speed and position.	63
5.6	Best global MediaPipe and YOLO pose-estimation settings. Lower composite score indicates better performance for the downstream stroke- phase task.	64
5.7	Best MediaPipe and YOLO performance by camera view.	65
5.8	Side-above-water wrist localization error for MediaPipe and YOLO. .	66
5.9	YOLO side-above-water LSTM test performance.	70
5.10	Combined classification report for the YOLO side-above-water LSTM model.	71
5.11	YOLO side-above-water LSTM accuracy by swimmer position.	72
5.12	Summary of manual stroke count validation results.	74
5.13	Manual validation results comparing manual stroke counts with auto- matic HMM/Viterbi stroke counts.	75

Chapter 1

Introduction

1.1 Background and Motivation

Competitive swimming is a highly technical sport where small differences in timing, body position, and stroke rhythm can affect performance. Swimmers compete for time over a fixed distance using four main strokes: freestyle, backstroke, breaststroke, and butterfly. Each stroke has its own movement pattern and phase structure, but each depends on repeated coordination between the arms, legs, torso, and breathing cycle. This thesis focuses on the analysis of freestyle as it is the most common competitive stroke and has a clear cyclic arm motion that can be analyzed over time. In competition, the difference between swimmers' times is often only milliseconds, making any competitive advantage tremendous. Tracking swimmer metrics such as stroke rate, stroke mechanics, pace, underwater time, and split times can optimize a race. Analytics in swimming can be the difference between a podium and a last-place finish.

In freestyle, each arm moves through a repeated sequence of four phases as seen in Figure 1.1. The hand enters the water and begins the catch, moves through the pull and push portions of propulsion, and then exits the water during recovery

before returning forward for the next stroke. These phases are important because they describe not only how many strokes a swimmer takes, but also how each stroke is performed. For coaches and swimmers, this type of phase-level information can provide more useful feedback than a final race time alone.

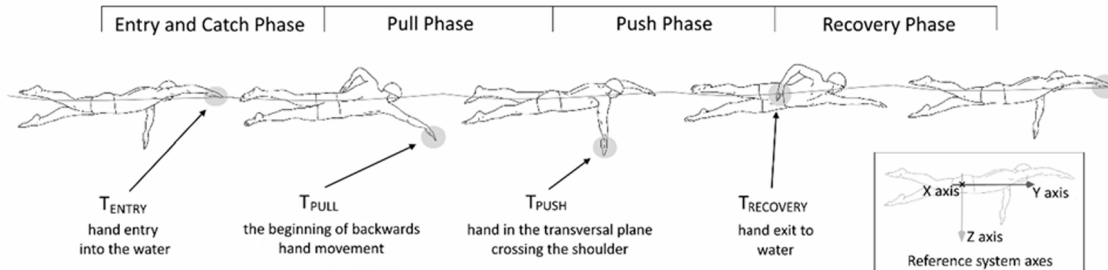


Figure 1.1: Arm stroke phases in front-crawl (classification proposed by Chollet et al., 2000) and graphical presentation of reference system axes [1].

Personalized analytics are increasingly important in sports because athletes need feedback that is specific, repeatable, and measurable. In swimming, useful metrics include stroke count, stroke rate, lap time, phase timing, and changes in technique across different conditions. Traditionally, many of these measurements are collected manually by coaches or analysts watching video. This process is time-consuming, subjective, and difficult to scale across many swimmers, videos, and training sessions. Automated swim analysis can help make these measurements more consistent and accessible.

However, automatically extracting meaningful kinematic information from swimming video is difficult. Water creates visual challenges that are not present in many land-based sports. Splashes, bubbles, turbulence, reflections, refraction, partial submersion, and poor contrast can all hide or distort the swimmer's body. Swimmers also rotate during freestyle, causing key joints such as the shoulders and wrists to appear differently depending on the camera angle. A joint may be visible in one frame and occluded in the next. The camera viewpoint also matters: side, front, aerial, water-level, and underwater views each emphasize different parts of the stroke cycle.

Because manually labeled swimming datasets are difficult and expensive to produce, this thesis investigates the use of synthetic data for video-based swim analysis. The synthetic dataset helps generate automatically generated pose and phase labels, reducing the need for frame-by-frame manual annotation. At the same time, synthetic data must be interpreted carefully because its usefulness depends on how well it represents real swimming motion, camera geometry, occlusion patterns, and water-related visual artifacts. In this work, the synthetic dataset is used as a controlled source of labeled training data, while its relationship to real-world swimming footage is considered as an important limitation and evaluation question.

The main objective of this thesis is to develop a video-based pipeline for detecting freestyle stroke phases and counting strokes without relying on manually labeled real-world training data. This thesis uses a synthetic swimming dataset with automatically generated pose and phase labels, compares computer vision (CV) pose-estimation models for extracting swimmer keypoints from video, and trains a temporal model to classify left- and right-arm stroke phases. By doing so, this work aims to make swimming analytics more interpretable, scalable, and less dependent on tedious manual annotation.

1.2 Problem Statement

Automated swimming analysis requires converting raw video into meaningful biomechanical measurements. This is challenging because raw video does not directly provide the swimmer’s joint positions, arm phases, or stroke count. A system must first estimate the swimmer’s pose, then interpret the motion over time, and finally determine when a complete stroke cycle has occurred.

Prior work has shown that stroke events and stroke rates can be estimated from video using computer vision and deep learning methods [3, 4]. Other work has shown that pose-based temporal models can support broader swimming analytics [5], and inertial-sensor research has demonstrated that wrist trajectory can be used to identify freestyle arm-stroke phases [6]. However, many of these approaches rely on manually labeled datasets, controlled environments, wearable sensors, or direct event detection rather than detailed frame-level phase classification.

This thesis addresses three connected problems. First, frame-level stroke-phase labels are expensive to create manually, especially in swimming videos, where the exact phase boundary can be ambiguous. Second, real computer vision pose-estimation models are imperfect in aquatic environments because keypoints may be missing, jittery, or incorrectly localized. Third, stroke counting is not simply a matter of detecting a single wrist position or peak. A stroke is a temporal cycle, so the system must recognize an ordered sequence of phases rather than isolated frame-level events.

To address these issues, this thesis treats stroke counting as a phase-sequence problem. The system predicts four freestyle phases for each arm: Recovery, Entry/Catch, Pull, and Push. A stroke is then counted when the predicted sequence completes the expected arm-cycle order. This phase-based approach provides a more interpretable intermediate representation than directly predicting a final stroke count from video.

The pipeline developed in this thesis uses synthetic data to automatically generate

Common Objects in Context 17 (COCO-17) style keypoints [7] and phase labels. The COCO17 style denotes the 17 human joint keypoint style derived from the COCO-pose dataset. Subsequently, a computer vision model is then used to extract keypoints from swimming videos. These keypoints are normalized and fed into a dual-arm Long Short-Term Memory (LSTM) model that predicts the phases of the left and right arms at each frame. Finally, temporal post-processing is used to smooth predictions and count completed strokes. This approach connects synthetic label generation, markerless pose estimation, temporal modeling, and rule-based stroke counting into a single video-to-stroke-count system.

1.3 Thesis Contributions

This thesis makes the following contributions to automated freestyle swimming analysis:

- A synthetic swimming dataset pipeline that automatically generates COCO17-style pose keypoints and frame-level freestyle phase labels without manual video annotation. The dataset supports multiple camera views and simulated swimmer/environmental conditions, and is validated in this thesis by testing whether its generated pose and phase labels can train a temporal model that generalizes across held-out synthetic videos.
- A comparison of YOLO and MediaPipe pose-estimation models for swimming video, using task-specific metrics such as joint localization error, missing-keypoint rate, temporal jitter, wrist-weighted performance, and a composite score designed for downstream stroke-phase classification. In the selected side-above-water view, YOLO achieved the best composite score of 2.3766 compared with MediaPipe’s 3.1007, motivating its use in the final pipeline.

- A YOLO-based side-above-water pose extraction pipeline that converts swimming video into an LSTM-ready dataset using keypoint alignment, MidHip normalization, x-coordinate rectification, and joint-presence masks. This pipeline allowed noisy real computer-vision keypoints to be fed to the same ML model as the clean synthetic COCO keypoints.
- A dual-arm LSTM phase-classification model that predicts Recovery, Entry/-Catch, Pull, and Push for the left and right arms at each frame from temporal pose windows. With perfect COCO pose inputs, the model achieved 94.69% overall frame-level accuracy on the held-out test set. With YOLO-extracted side-above-water keypoints, the model achieved 83.63% overall frame-level accuracy, showing that the method remains effective under realistic pose-estimation noise.
- A temporal post-processing and stroke-counting method that uses HMM/Viterbi smoothing and rule-based phase-cycle counting to convert frame-level phase predictions into stroke counts. In manual validation on 20 selected test videos, the full pipeline achieved a total mean absolute error of 0.80 strokes, exact total-count accuracy of 35.00%, and within-one total-count accuracy of 85.00%.

Together, these contributions show that synthetic pose and phase labels can be used to train a temporal model for freestyle phase classification, and that real computer vision keypoints can be incorporated into the same pipeline with measurable performance. The results also show where the pipeline succeeds and where it remains limited: Recovery and Entry/Catch were classified most reliably, while Pull was the most difficult phase, especially when using noisy YOLO keypoints. The phase-based design makes the final stroke count more interpretable because errors can be traced back to pose extraction, phase classification, temporal smoothing, or counting logic.

Chapter 2

Related Work

Across prior work in swimming analysis, a common challenge is that any motion is often hidden, distorted by water, or costly to label by hand. Studies show that both computer vision and sensor-based methods can provide useful swimming metrics. Still, many methods use shortcuts such as manually labeled datasets, controlled environments, wearable sensors, or custom processing steps. This project aims to provide a different approach by creating a fully automatic pipeline. It uses a synthetic dataset as an input that automatically generates labels, then applies computer vision and temporal modeling to estimate freestyle stroke phases and stroke counts from video.

2.1 Pose Estimation in Aquatic Environments

Early work combining swimming and computer vision focused on using classical image-processing techniques to detect swimming motions from underwater video. Dubois et al. [8] investigated how digital image processing techniques such as segmentation, boundary detection, and deformable body models could support biomechanical analysis in swimming videos. Their work emphasized that underwater video

is often degraded by blur, bubbles, cavitation artifacts, camera motion, and poor contrast. Furthermore, to detect a swimmer’s joints and body points, they used methods such as Normalized Cuts segmentation, boundary detection, and pictorial/deformable models. Their results demonstrate the feasibility of identifying and estimating a swimmer’s limb segments solely through image processing. Importantly, they argued that once body configuration can be estimated, higher-level swimming metrics such as stroke count, stroke rate, and arm-stroke phases can be extracted.

This earlier work is relevant because it frames swimming analysis as a body-motion extraction problem rather than only an image-classification problem. Hence, rather than classifying swimming actions directly from video, they suggest that joint and pose extraction data, fed to a classifier, yield the best results.

As is common in modern pose estimation research, swimmer pose estimation has moved away from image processing toward CV techniques. Even if there is a change, the previous issues mentioned by Dubois et al. [8] remain. Current research using modern pose-estimation CV models continues to find that aquatic pose estimation is affected by optical distortion, turbulence, bubbles, occlusion, and non-canonical body orientations [9]. Current research by Caputo et al. [9] specifically discusses the optimization of CV models to address these issues. They used the two most common open-source computer vision models, Ultralytics YOLO [10] and Google MediaPipe [11], to benchmark and tune them for swimmer pose estimation.

This research concluded that generic pose metrics used in land-based sports and in generic computer vision techniques are not always sufficient for aquatic sports analysis. Standard averages can hide the fact that some landmarks are much more important than others. For example, hips and shoulders are usually easier to localize because they lie on the surface of the water, while wrists, hands, ankles, and feet are smaller, faster-moving, and more likely to be occluded. Caputo et al. [9] proposed

new evaluation metrics, including anatomical tolerance thresholds and normalized localization accuracy, to better measure whether a pose is accurate and useful for swimmer analysis. A key contribution of their research is their use of joint-weighted metrics. This places a higher “weighted score” onto harder-to-localize joints, such as the wrist and feet, allowing a model to be correctly evaluated for its applicability towards joint detection in water environments

2.2 Video-Based Stroke Detection

Creating a dataset with a target variable, such as the stroke type or phase from swimming videos, is commonly done through manual annotation. This is a slow and tedious process that requires a human to watch every video frame and mark every joint or event. Instead of this process, Victor et al. [3] proposed an automated deep learning approach for stroke detection. Essentially, they used a convolutional neural network (CNN) to detect in which frames a swimmer’s hand enters the water and use this to classify a stroke “event”. Instead of training the CNN with hard labels such as stroke = 1 or no stroke = 0, they turn the labels into a smooth 1D signal. Each stroke becomes a peak in the signal, and nearby frames have values close to it. This helps because the exact stroke frame can be ambiguous, even for humans. Instead of treating event detection as a binary classification task, they turn it into a signal regression problem. That makes the learning problem smoother, more stable, and more tolerant of small labeling errors. This allows their CNN architecture to more accurately learn the target variable for stroke detection. Furthermore, they compared multiple dataset types, including single-frame input, early frame fusion, and 3D convolution. Using videos with a frame rate of 30 frames per second, the 3D convolution model achieved the best swimming F-score of 0.922 at a 3-frame tolerance, but early fusion also performed well and was computationally simpler.

Instead of predicting a stroke phase label for every frame (e.g., Recovery, Entry/-Catch, Pull, and Push), Victor et al. predict a continuous signal in which peaks correspond to important stroke events. This is a simpler event-detection version compared to the phase-classification problem in this paper.

The main limitation is that the approach proposed by Victor et al. [3] only detects stroke events, not full stroke phases or detailed body mechanics. This method also uses a preprocessed video in which each lane is cropped to focus on a single swimmer. Hence, this method does not solve the full real-world problem of detecting swimmers, estimating pose, handling multiple swimmers, and simultaneously classifying detailed stroke phases.

Other relevant research includes that of Woinoski and Bajić [4]. They proposed a system for estimating swimmer stroke rates from real overhead race video of the 2019 TYR Pro Swim Series, where the camera may move or capture only part of the pool. Their pipeline first trained YOLOv3 models to detect swimmers in overhead race video. Each swimmer was classified into one of six states: on-blocks, diving, swimming, underwater, turning, or finishing. Once swimmers are detected, SORT tracking combines YOLO detections, a Kalman filter, and the Hungarian algorithm. This lets the system follow a swimmer through the video and create a smaller “sub-video” for each swimmer. For each swimmer sub-video, a CNN predicts a continuous stroke signal called an s-value. An s-value of 1 represents the top of a stroke, and 0 represents the bottom of a stroke. The signal is then smoothed with a Butterworth filter, thresholded, and converted into stroke peak locations. Stroke rate is computed from the time between neighboring peaks. Essentially, Woinoski and Bajić avoid treating stroke detection as a simple binary classification problem. Instead, they turn the stroke cycle into a smooth, continuous-time signal that detects peaks in it in a similar way to Victor et al.. That makes the learning problem easier and more tolerant

of small timing errors in the labels. Their results showed that their models, using videos with a frame rate of 30 frames per second, can locate stroke peaks within about 2–3 frames on average, though the standard deviation is around 4 frames. The main contribution is that the paper extends earlier stroke-signal work into a more complex video-to-stroke detection pipeline. It still uses CNNs, however, on the video itself rather than the swimmer’s pose keypoints. Furthermore, their dataset, which was created by generating 346 swimmer sub-videos, consisted of manually generated labels, including swimmer-detection annotations and stroke-rate labels. This means that research into automatic dataset annotations, especially on joint keypoints, remains a novel aspect of swimming analysis research.

Recent work has moved toward more complete vision-based swimming analytics systems. Tran et al. [5] introduced Dronaquatics, a drone-based system for analyzing swimmers using only overhead video. This recent research now focuses on actually estimating a swimmer’s body pose from video to compute performance metrics such as stroke type, stroke count, instantaneous speed, and lap time.

As mentioned, a drone is used to record the dataset. Unlike to fixed cameras, which can have a limited field of view, occlusions, and perspective distortion, a drone provides a flexible overhead view that can follow the swimmer and better avoid those issues. The Dronaquatics researchers tuned a YOLO model on their manually annotated dataset to enable it to detect a swimmer’s bounding box and 17 COCO-style keypoints using aerial footage. This joint data was then fed to an RNN/GRU model (similar to LSTM). For stroke classification, Tran et al. [5] report precision of about 0.92 and recall of about 0.93 across stroke classes. For speed estimation, the median error is below 4%, or under 0.05 m/s. For lap time, the median error is around 0.03 seconds. Stroke count errors are typically under one stroke per lap.

One important design choice is that Tran et al. [5] train the stroke classifier

on pose sequences rather than raw video, as seen in previous research. For their RNN model, a 90-frame window size was chosen for the input. Each frame contains a sequence of 17 COCO keypoints with x, y, and confidence values. This research provides a good justification for why pose-based temporal models are a reasonable approach for swimming analysis.

Frame rate is an important practical consideration for video-based swimming analysis because stroke events and phase transitions occur over short time intervals. Prior video-based swimming studies have reported successful stroke detection at 30 frames per second [3, 4]. Camera equipment commonly also uses a frame rate of 60 frames per second. At 30 fps, each frame represents approximately 33.3 ms, while at 60 fps each frame represents approximately 16.7 ms. Therefore, 60 fps provides finer temporal resolution for identifying phase boundaries, wrist motion changes, and stroke-cycle transitions. Although even higher-speed cameras may be useful for detailed biomechanical analysis of very fast events, the previously described research shows that 60 fps is a reasonable and sufficient frame rate.

2.3 Inertial Sensors for Stroke Phase Detection

Not all swimming analytics systems rely on video. Cortesi et al. [6] studied the use of wearable inertial measurement unit (IMU) sensors instead of video and computer vision models to detect freestyle stroke phases. The benefit of this approach is that it avoids many drawbacks of visual systems, including optical distortion, turbulence, bubbles, and occlusion. An inertial sensor can measure motion consistently regardless of whether the limb is above or below the water. The main drawback, however, is that the swimmer must wear five inertial sensors placed on the thorax, upper arms, and forearms. These sensors can add drag, cause discomfort, and interfere with a swimmer's natural movement. Ideally, swimming analytics would not require the

swimmer to wear any sensors, and all data would come from an external system such as video. However, video data remains more vulnerable to occlusion and visual distortion than IMU sensor data. Therefore, each approach requires tradeoffs between accuracy, comfort, and practicality.

Cortesi et al. [6] focused on automatically identifying the four main front-crawl arm-stroke phases: Entry/Catch, Pull, Push, and Recovery. They developed a rule-based algorithm, rather than a deep learning model, to identify key phase events from the three-dimensional wrist trajectory. This is important because it supports the central biomechanical assumption underlying this project: wrist trajectory contains enough information to distinguish stroke phases. Their rule-based method detects the main phase transitions as follows:

- Recovery start / hand exit: detected from the forearm angle signal.
- Entry: detected when the wrist slows down near the end of recovery.
- Pull: detected when the wrist begins accelerating backward.
- Push: detected when the wrist reaches its maximum underwater depth.

The IMU-based phase detections were compared against manually annotated underwater video, which served as the ground truth. Cortesi et al. found that their algorithm performed similarly at both 75% and 100% of each trial swimmer’s maximal velocity over 25 meters. They also found that changes in arm coordination style did not significantly affect phase detection. This suggests that wrist-trajectory-based phase detection is reasonably robust across different front-crawl speeds and coordination patterns.

This phase-based view is also useful for stroke counting. Stroke counting requires identifying repeated arm cycles, but raw joint trajectories alone can contain noise, occlusions, and local motion peaks that do not correspond to complete strokes.

Therefore, this project first models the stroke as a sequence of biomechanical phases: Entry/Catch, Pull, Push, and Recovery. Prior work has shown that these phases can be detected from wrist trajectory events and used to detect freestyle phases [6]. By predicting phases frame by frame, each stroke can be counted as a completed phase sequence rather than an isolated motion peak. This makes the stroke count more robust and interpretable.

2.4 Research Gap

Prior work shows that swimming analytics can be extracted from video, pose data, and wearable sensors. However, several gaps remain. First, many video-based methods focus on stroke detection or stroke-rate estimation rather than detailed stroke-phase classification. For example, Victor et al. and Woinoski and Bajić predict smooth stroke-cycle signals and use signal peaks to identify stroke events or estimate stroke rate [3, 4]. While this is effective for counting strokes, it does not directly identify the biomechanical phases that make up each stroke cycle, such as Entry/Catch, Pull, Push, and Recovery. As a result, these methods provide limited information about the structure of the swimmer’s technique.

Second, recent vision-based systems have begun using pose sequences for swimming analytics, but they still rely heavily on manually annotated real-world datasets. Dronaquatics, for example, fine-tunes pose estimation models on manually labeled drone footage and uses pose sequences to classify stroke type, estimate speed, count strokes, and calculate lap time [5]. This demonstrates the value of pose-based temporal modeling, but it does not eliminate the need for expensive manual annotation. In swimming, this labeling burden is especially difficult because important joints are frequently occluded by water, splashes, bubbles, and body motion.

Third, sensor-based methods accurately detect stroke phases from wrist trajeceto-

ries, but they require wearable devices. Cortesi et al. demonstrated that IMU-based wrist trajectories can identify Entry/Catch, Pull, Push, and Recovery with strong agreement compared to video-based ground truth [6]. However, this approach depends on sensors attached to the swimmer, which can add drag, cause discomfort, and interfere with natural motion. Therefore, while IMU systems provide strong biomechanical evidence for using wrist trajectory to detect phases, they are not ideal for unobtrusive video-based swimming analytics.

These limitations create the main gap this project addresses. Existing work has either focused on stroke-event detection from video, relied on manually labeled real-world datasets, or used wearable sensors for detailed phase detection. There remains a need for a pipeline that can connect these approaches: automatically generated pose and phase labels from a synthetic swimming dataset, computer vision models to estimate pose from video, and temporal modeling to classify freestyle stroke phases frame by frame. This project addresses that gap by treating stroke counting as a phase-sequence problem rather than only a peak-detection problem. By predicting Entry/Catch, Pull, Push, and Recovery over time, the system can count strokes as completed biomechanical cycles, making the final stroke count more interpretable and more closely tied to swimming technique.

Chapter 3

SwimXYZ Dataset and Automatic Phase Labeling

3.1 SwimXYZ Dataset

Researchers from the Centrale Supélec, Centrale Nantes, and Université technologique de Compiègne created a synthetic dataset titled SwimXYZ [12]. SwimXYZ is a large-scale synthetic dataset designed to address the lack of labeled swimming data for computer vision tasks such as pose estimation, motion capture, and stroke analysis. It contains 3.4 million fully annotated frames (11,520 synthetic videos), making it the largest publicly available swimming dataset. This dataset uses the Unity engine to generate 3D models of 60 unique synthetic swimmers, each swimming each of the 4 strokes in an outdoor 25-meter pool. SwimXYZ purposefully incorporates variations in technique, speed, and phase timing into each swimmer and individual stroke repetitions. Furthermore, the Unity environment itself incorporates adjustable lighting, surface/bottom color gradients, water transparency and refraction, waves and under-water distortions, particle systems for splashes, and multiple camera angles (5 preset views). Regardless, each video is 5 seconds long at 60 fps, with the swimmer already

swimming at the start.

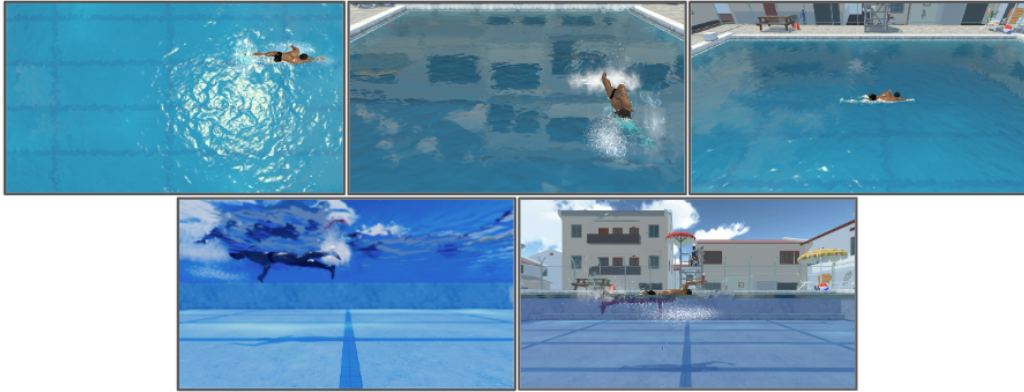


Figure 3.1: Sample images from SwimXYZ for freestyle motions from the various viewpoints. The top left, middle, and right are the Aerial, Front, and Side Above Water views respectively. The bottom left and right are the Side Underwater and Side Water Level Right views respectively.

Figure 3.2 demonstrates the various hyperparameters used by SwimXYZ. First is the stroke, which contains all 4 standard swimming strokes. Next are the different viewpoints simulated from a fixed monocular camera on the side of a 6-lane 25-meter pool. Subject appearance denotes the unity of swimmer models’ skin and muscle completion. A higher value indicates a darker, more muscular complexion. These are synthetic appearance controls, not real measured biological values, and help add visual diversity to the existing 3D swimmer models. It is important to note that the models only consist of male 3D renderings and no other genders.

Furthermore, the speed and starting position denote other swimmer attributes in the physical pool environment. On the other hand, the waves and lighting denote attributes associated with the pool. The wave values dictate the simulated natural ambient turbulence in the pool water around the swimmer. The lighting values dictate the position and angle of the simulated sun rays onto the pool water, recreating the various possible refraction and reflection patterns in an outdoor environment. These hyperparameters introduce variability, enabling the generation of over 11,520 synthetic videos. This allows for greater flexibility in understanding which position

data and camera angles are key to modeling the freestyle stroke. Conversely, the use of and experimentation with a real dataset would be constrained by practical recording conditions.

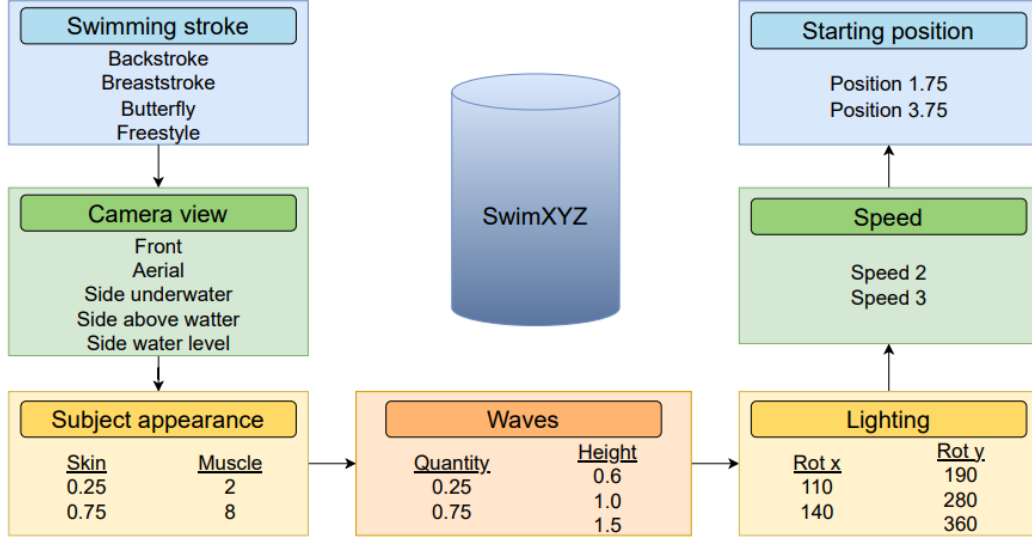


Figure 3.2: Structure of the possible hyperparameter settings for each video

In addition to the variability in swimmer and environment modeling, the actual swimming motion is also varied. Having a single swimming motion would lead to a weak dataset for training deep learning models. Using the same motion pattern for each stroke would lead to overfitting and prevent models from generalizing from synthetic to real-world applications.

Random variations in the swimming motion are introduced using a GANimator which is a generative model that generates new motions given a short motion sequence [13]. It uses a series of generative and adversarial neural networks to generate motions that resemble the core elements of a given motion, using only a single motion sequence. This model was shown to work in a number of applications such as crowd simulation and style transfer. The SwimXYZ researchers used this model architecture to train 4 GANimators independently on each of the 4 strokes. This created novel 3D joint sequences, which were then passed into the 3D Unity simulation environment. Figure

3.3, shows the full pipeline for generating the videos.

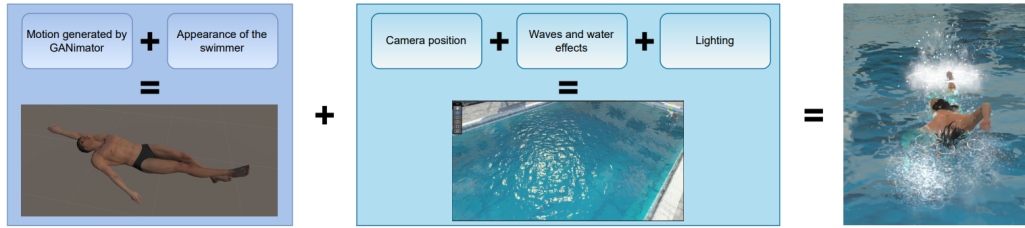


Figure 3.3: SwimXYZ pipeline for generating swimming videos

Combining these unique swimming motions with the previously mentioned set of hyperparameters creates a dataset with unique video sets. The hyperparameters given to create the view set (front, aerial, side underwater, side above water, and side water level) are not repeated. Between the videos in a given view set, the hyperparameters remain the same. Each video also contains corresponding 3D and 2D joint annotations in a global coordinate frame and a pelvis-centered coordinate frame (with the origin at the swimmer’s midpoint between the left and right pelvis joints). These annotations are available in the COCO17 [7], COCO25 (also referred to as Body25) [14], and base formats.

The Microsoft Common Objects in Context (MS COCO) dataset is widely used for image classification and detection. It was developed in 2015 to compete with the ImageNet dataset by containing fewer total object classes but more instances per class. In 2017, the COCO-Pose dataset was developed [7]. This is a specialized subset of the COCO dataset used to train deep learning models for human pose and keypoint detection. The COCO pose format of this dataset (COCO17) consists of a standardized 17-keypoint skeleton, including the nose, eyes, ears, shoulders, elbows, wrists, hips, knees, and ankles. Each keypoint consists of 3 values: x and y coordinates, and a visibility flag (v). This dataset is what many common computer vision models, including YOLO, are trained on. This means the models’ output data is structured around the standard 17-point COCO human pose format and predict the same ma-

major joints—shoulders, elbows, wrists, hips, knees, and ankles—utilized in the COCO Keypoints definition. COCO25 also referred to as BODY25, expands on the original COCO17 format and include 25 keypoints as opposed to 17. Both the COCO17 and COCO25/Body25 are common annotation formats used in modern computer vision research.

The base format refers to the researcher’s most detailed format with a full 48-joint skeleton. In SwimXYZ, the base coordinate system is the most precise. All annotation types technically contain both 3D and 2D joint annotations. However, for the COCO formats, the 3rd-dimensional axis (z-axis) is set to 0, in effect rendering the 3D annotations 2D. This is to emulate the output of a computer vision model that infers 2D joint positions from a 2D image. Due to the increased joint precision from the 3D base annotations, they were used to explore the dataset structure and to begin phase analysis.

The base coordinates were used to explore the dataset structure. For freestyle analysis, the stroke phase and count are determined by the swimmer’s arm position, not the kicking action. Therefore, the 3D upper-body joints must be examined. Figure 3.4 describes the 3D coordinate axes used. For the base coordinate system, the Z-axis refers to the forward and backwards direction of the swimmer in the water. It is the axis from their head to their feet while they lie flat on the water. The Y-axis is the vertical world axis. It can be described as the up and down axis in the water, or also as from the swimmer’s stomach to the back if they lie flat on the water. The X-axis is the lateral axis, meaning the left-right direction for the swimmer. It is parallel to the pool’s water level. Figure 3.5 shows a sample Base annotation from an individual sample frame, including the relevant upper body joints. Specifically, it shows the relevant joints as they appear on the right arm. Based upon the description of these axes, the XY plane is as if looking at the swimmer from behind as they swim. The

XZ plane would be the top view, while the ZY plane is the right side view.

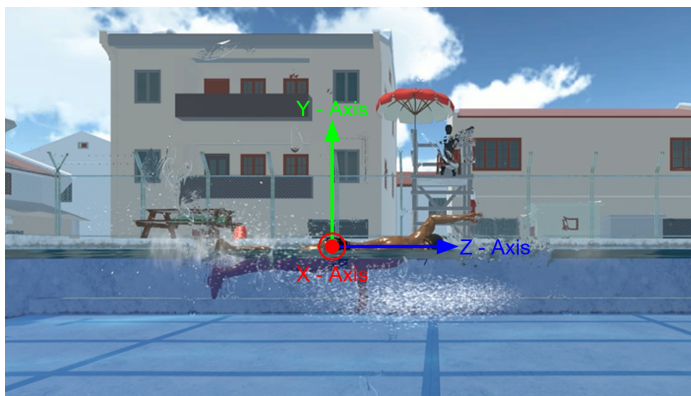


Figure 3.4: Side Water view of swimmer with overlaid x,y,z coordinate system used in SwimXYZ dataset

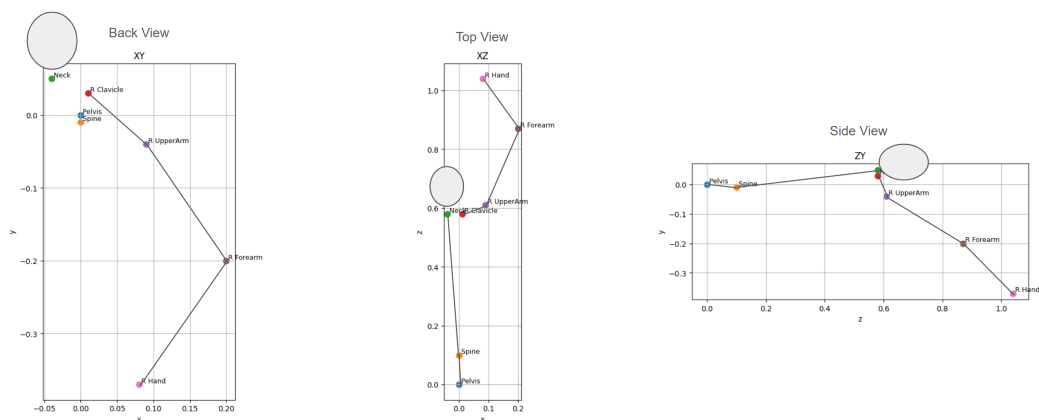


Figure 3.5: Cross sections of the swimmer's right arm. The grey ellipse represents the swimmer's head. Left: view from behind swimmer. Middle: top view of swimmer. Right: right side view of swimmer

Figures 3.6 and 3.7 show the same sample annotations over the span of the entire 5-second video. As these base annotations are pelvis-centered, the x,y,z values are all relative to the pelvis joint. The Figure shows how this allows for the upper-body joints to become cyclical. Note how the pelvis joint is constant at 0.

Another important aspect of the SwimXYZ dataset is the swimmer leaving the frame. Each viewpoint is fixed in place in the Unity simulation environment. This means that, depending on the swimmer’s speed and starting position, the swimmer

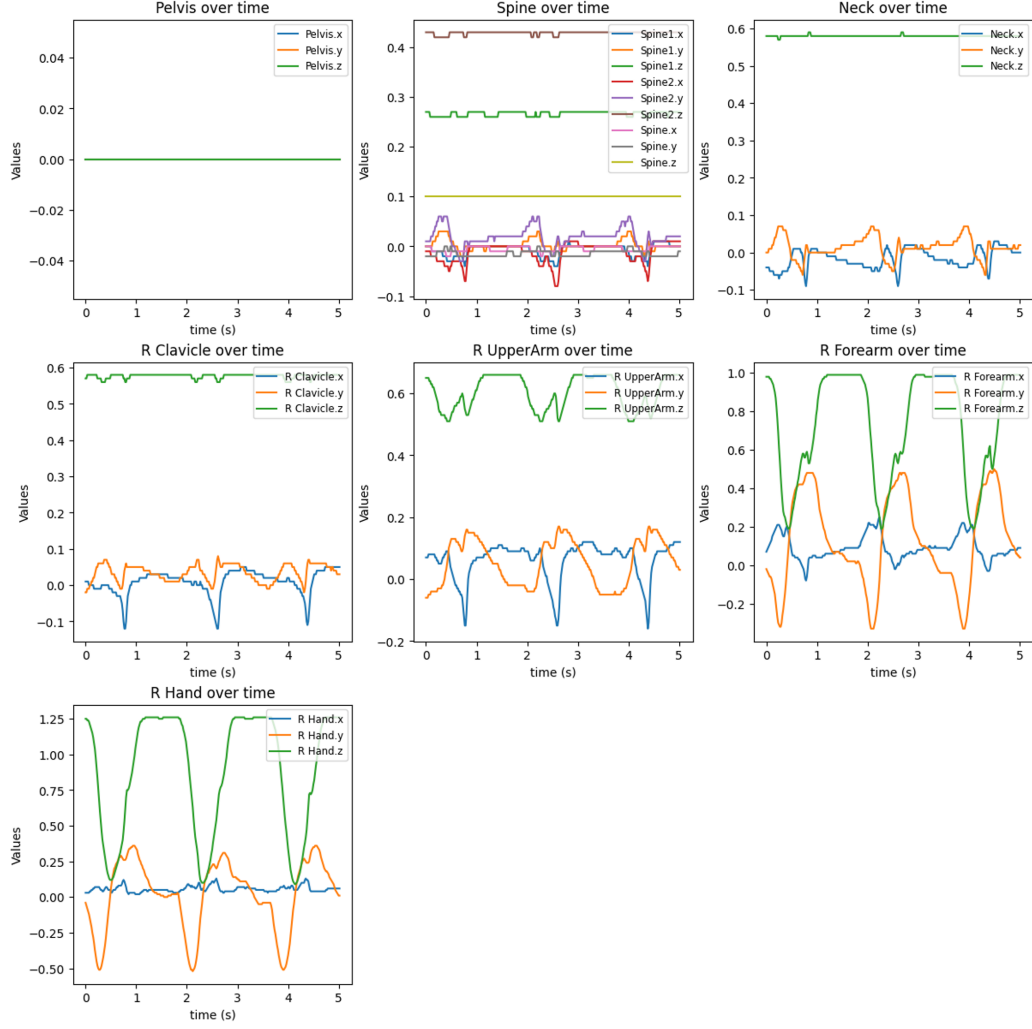


Figure 3.6: The Base data annotations over a full 5 second video

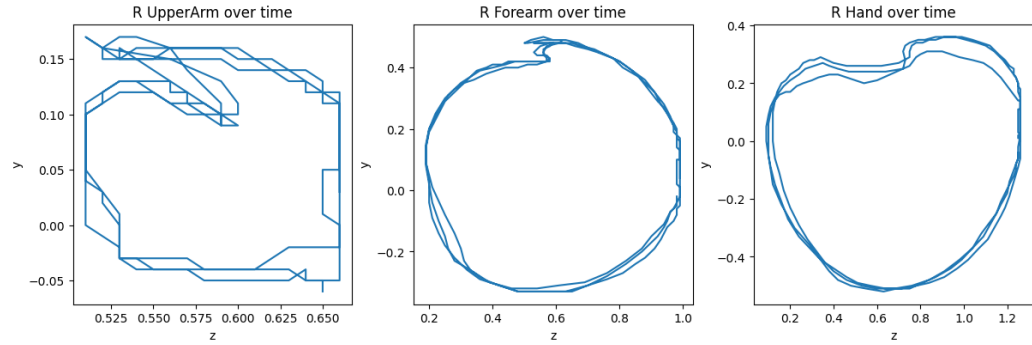


Figure 3.7: The cyclic motions of relevant joints in the ZY plane over a 5 second sample video from the Base annotation set

may go outside the camera’s field of view within the 5-second video. The SwimXYZ researchers decided to address this issue by having the swimmer instantaneously switch directions once their full body leaves the frame. This means that once the swimmer has exited the frame, they do a full 180-degree flip and swim back in the direction from which they came. This was done to ensure that the videos contain only swimming motions, with no push-offs or flip turns. Unfortunately, this flip in direction appears to be random and lacks a flag or label in the joint annotations. This discontinuity creates an issue when doing any calculations on this data. The sample video data in Figures 3.8 and 3.9 show how the flip creates an instantaneous discontinuity at 4.2 seconds for this particular video. Multiple methods were tested to detect and remove discontinuities in the videos, including calculating derivatives. However, the way the SwimXYZ dataset was created can be leveraged to address this.

In the base coordinate system, the swimmer swims forward and backwards only along the Z-axis in all videos. Similarly, for all Coco-style annotations, the swimming motion moves across the X-axis. The simplest method is therefore to take the absolute value of each axis. Figures 3.8 and 3.9 show the effect of the rectification pipeline.

3.2 Automatic Phase Labeling

As the SwimXYZ dataset lacks manually annotated stroke-phase labels, there is a need for stroke-phase labeling. In this thesis, an automatic labeling procedure was developed to provide frame-level supervision for the recurrent LSTM phase-detection model. The goal of this step was to convert each freestyle sequence into two synchronized label streams, one for the left arm and one for the right arm, where all valid frames are assigned to one of four freestyle phases: Recovery, Entry/Catch, Pull, or Push. These labels were first generated from the synthetic 3D base data, which provides the most complete and physically consistent joint information, and were then

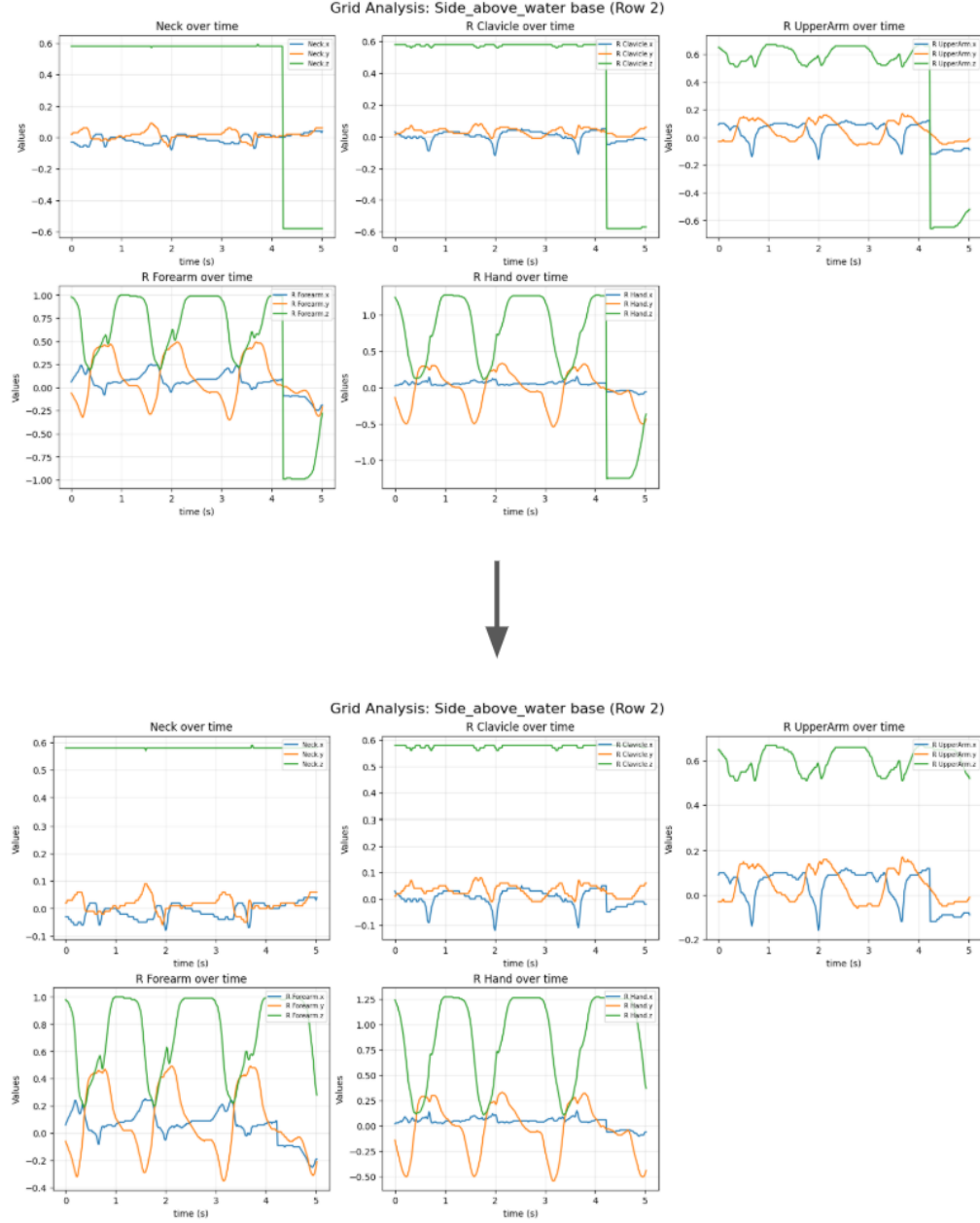


Figure 3.8: The before and after of the data preprocessing on the base data for a sample video. The data already normalized by the hip joint. The absolute value of the Z-coordinate is taken.

transferred to the corresponding COCO and CV-derived pose datasets.

This automatic labeling method provided a practical way to create a complete frame-level phase dataset without manual annotation. Although the labels are heuristic rather than human-labeled ground truth, they are derived from the clean 3D

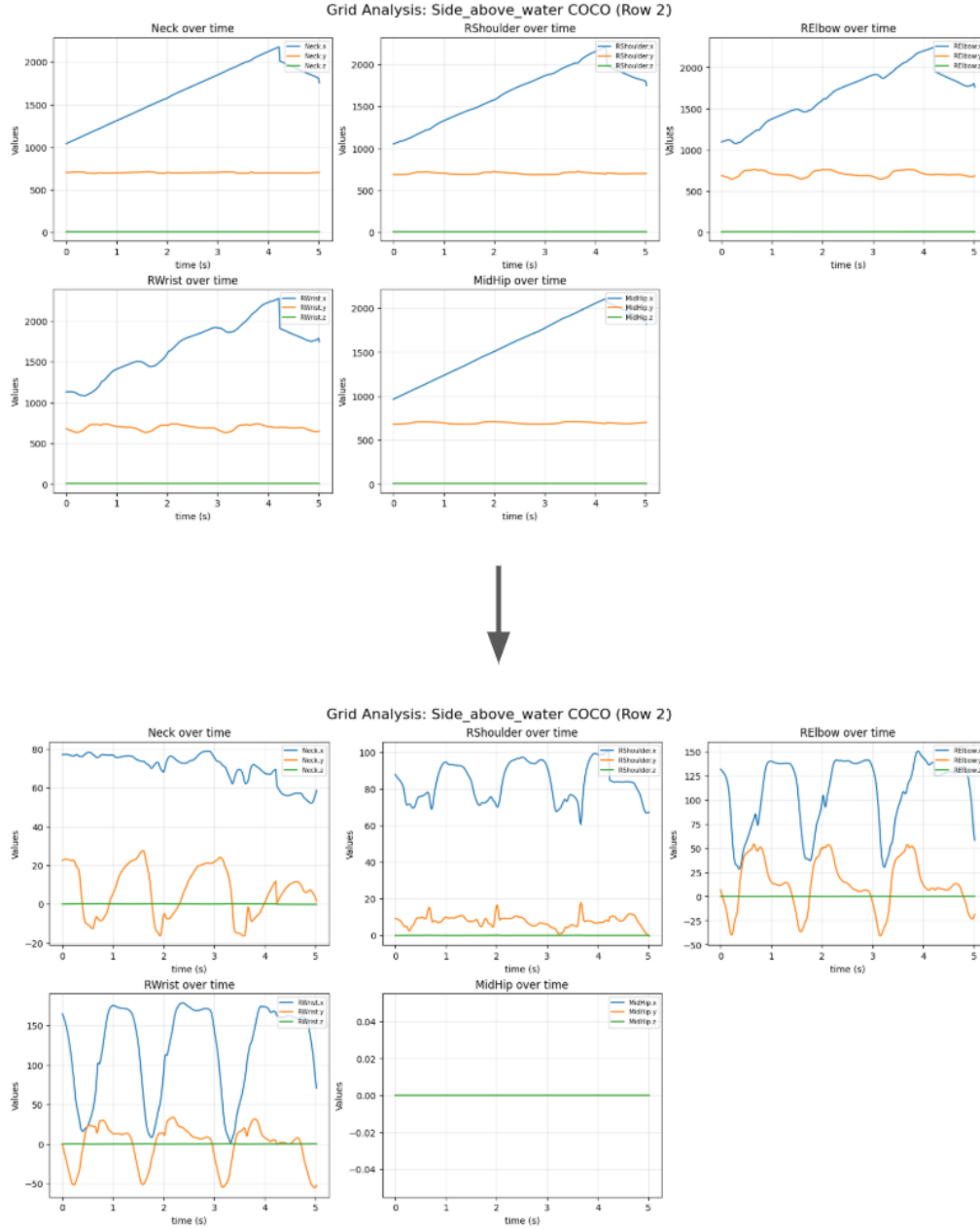


Figure 3.9: The before and after of the data preprocessing on the COCO data for a sample video. The data is normalized by the hip joint and the absolute value of the X-coordinate is taken.

synthetic skeletons and then consistently transferred to the noisier 2D pose representations. This allowed the project to train and compare LSTM models using base data, COCO keypoints, and YOLO-extracted keypoints, while maintaining a shared definition of the freestyle stroke phase across all input formats.

The phase-labeling method was designed around the observation that freestyle arm motion follows a repeated cyclic pattern that can be approximated using hand position and arm elevation. For each swimmer, view, and arm, three signals were extracted from the 3D base annotations: the hand y-coordinate, the rectified hand z-coordinate, and the hand elevation angle. Figure 3.10 demonstrates how flexion can be described in the human body in both the sagittal and scapular planes.

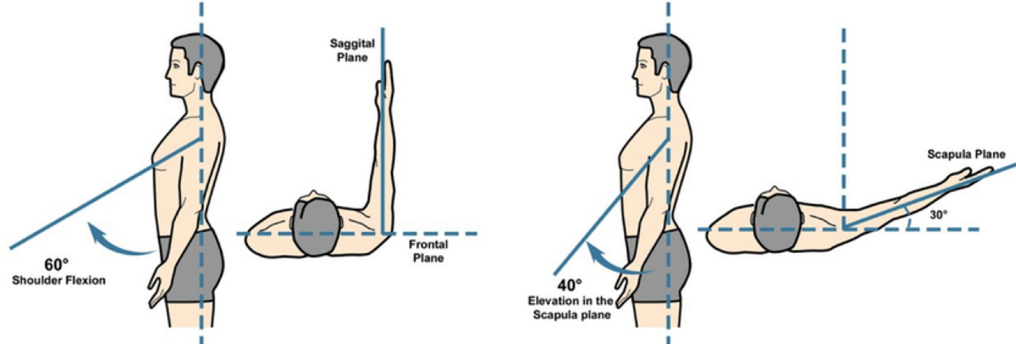


Figure 3.10: Shoulder flexion angle compared to the Sagittal and Scapular Planes [2].

The hand elevation angle was calculated as the angle between the torso axis and the arm vector in the zy-plane. The torso axis was defined from the pelvis to the neck, while the arm vector was defined from the clavicle/shoulder region to the hand. This made the angle more robust to the global swimmer position because it describes the arm motion relative to the swimmer's body rather than only relative to the camera or pool coordinate system.

The labeling algorithm was developed by detecting key transition events in the swimmer's arm cycle. Figure 3.11 shows a sample swimming motion from the base hand joint from a random video in the dataset. First, local lows in the rectified hand z-coordinate are detected using peak detection on the negated z signal. These events are treated as the start of the Recovery phase. Second, the algorithm detects the start of a high, flat region in the rectified z signal. A frame is considered a plateau start when the z-coordinate exceeds a threshold, and the z-velocity becomes small,

indicating that the hand has reached a relatively stable high position. These events are labeled as the start of Entry/Catch. The position and velocity threshold values seen in Algorithm 1, where chosen through manual error trial. Third, the algorithm searches forward from each plateau event to find the first frame where the hand elevation calculated from the arm and torso vectors reaches 45 degrees. This frame marks the start of the Pull phase. Finally, local minima in the raw hand y-coordinate are detected and used as the start of the Push phase. The pseudocode in Algorithm 1, further explains this logic.

Algorithm 1 Automatic freestyle arm-phase labeling

Require: Raw and rectified base hand trajectories, hand elevation angle $\theta(t)$

Ensure: Frame-level phase labels for one arm

- 1: Extract rectified hand depth $z(t)$ and raw hand height $y(t)$
 - 2: Initialize all frame labels as *None*
 - 3: Initialize an empty event list E
 - 4: Detect local minima in $z(t)$ using peak detection on $-z(t)$
 - 5: Add these frames to E as *Recovery* events
 - 6: Compute hand depth velocity $v_z(t) = \nabla z(t)$
 - 7: Detect plateau-start frames where:
$$z(t) > 1.1, \quad |v_z(t)| < 0.05, \quad v_z(t-1) \geq 0.05$$
 - 8: Add these frames to E as *Entry/Catch* events
 - 9: **for** each *Entry/Catch* event at frame t_e **do**
 - 10: Find the first later frame t_p where $\theta(t_p) \geq 45^\circ$
 - 11: **if** t_p exists **then**
 - 12: Add t_p to E as a *Pull* event
 - 13: **end if**
 - 14: **end for**
 - 15: Detect local minima in $y(t)$ using peak detection on $-y(t)$
 - 16: Add these frames to E as *Push* events
 - 17: Sort E by frame index
 - 18: **for** each event (t_i, p_i) in E **do**
 - 19: Let t_{next} be the next event frame, or the final frame if this is the last event
 - 20: Assign phase p_i to all frames from t_i to $t_{next} - 1$
 - 21: **end for**
 - 22: **return** frame-level phase labels
-

After these events are detected, they are merged into a single chronological event

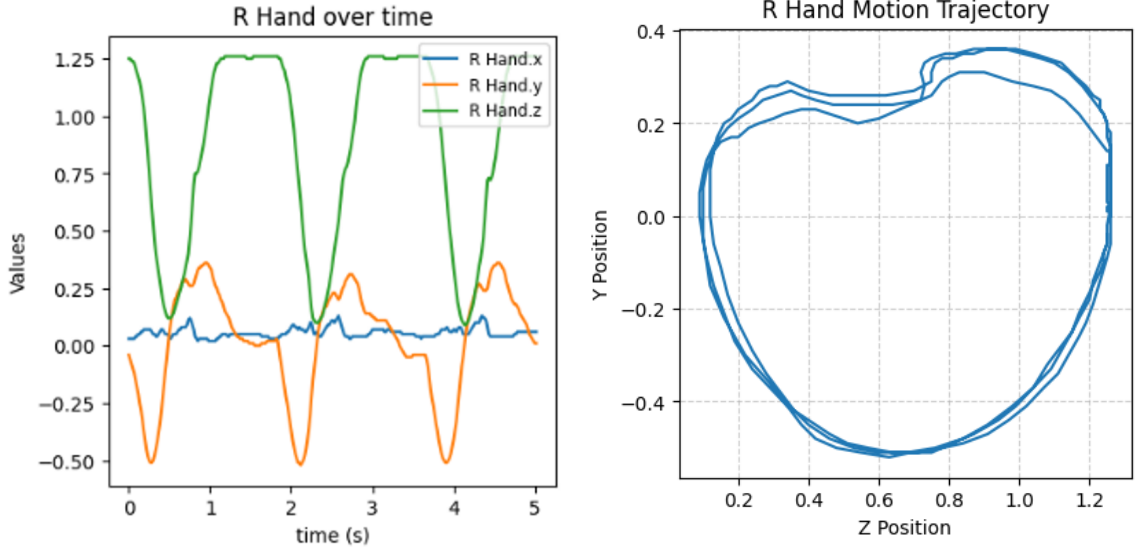


Figure 3.11: The right hand motions over a 5 second sample video from the Base annotation set. The left plot displays the right hand X, Y, and Z motions while the right plot displays the cyclic hand motion in the ZY plane for the same video

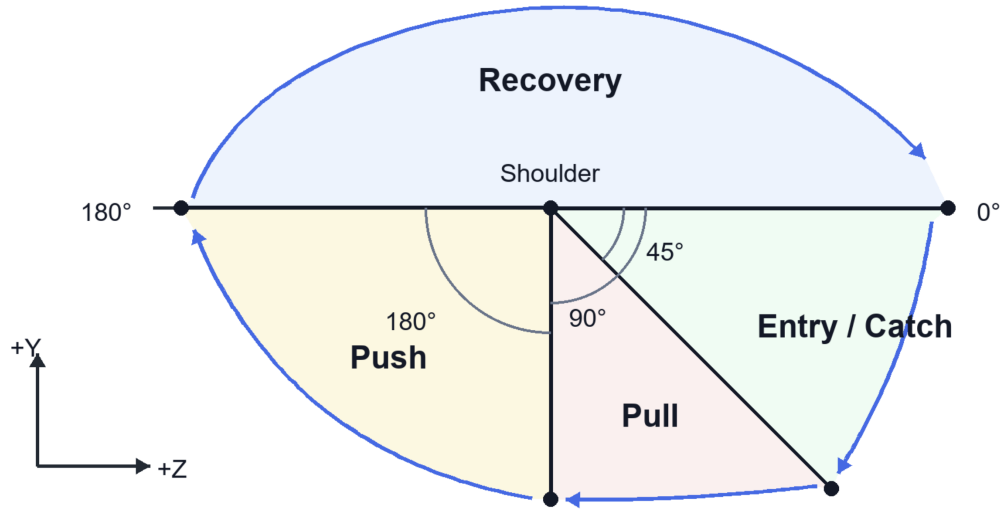


Figure 3.12: How the shoulder elevation angle of a swimmer corresponds to the freestyle stroke phase. This is a side view perspective of the swimmer (ZY plane), meaning theoretical swimmer's head rests at 0 degrees while the feet lay at 180 degrees relative to the swimmers right shoulder joint.

list. The label assigned to each event is carried forward until the next event. For example, if a Recovery event is followed by an Entry/Catch event, all frames between them are labeled Recovery. Frames before the first detected event are assigned a temporary value of None; these frames are later excluded from window extraction

during dataset preparation. This process produces a frame-level phase sequence for each arm independently, allowing the left and right arms to be out of phase as expected in freestyle swimming. Hence, this creates two independent target variables for the set of input swimmer joint annotations. Figure 3.13 is a sample of how the resulting phase labels look as compared to the base annotation and calculated shoulder elevation angle.

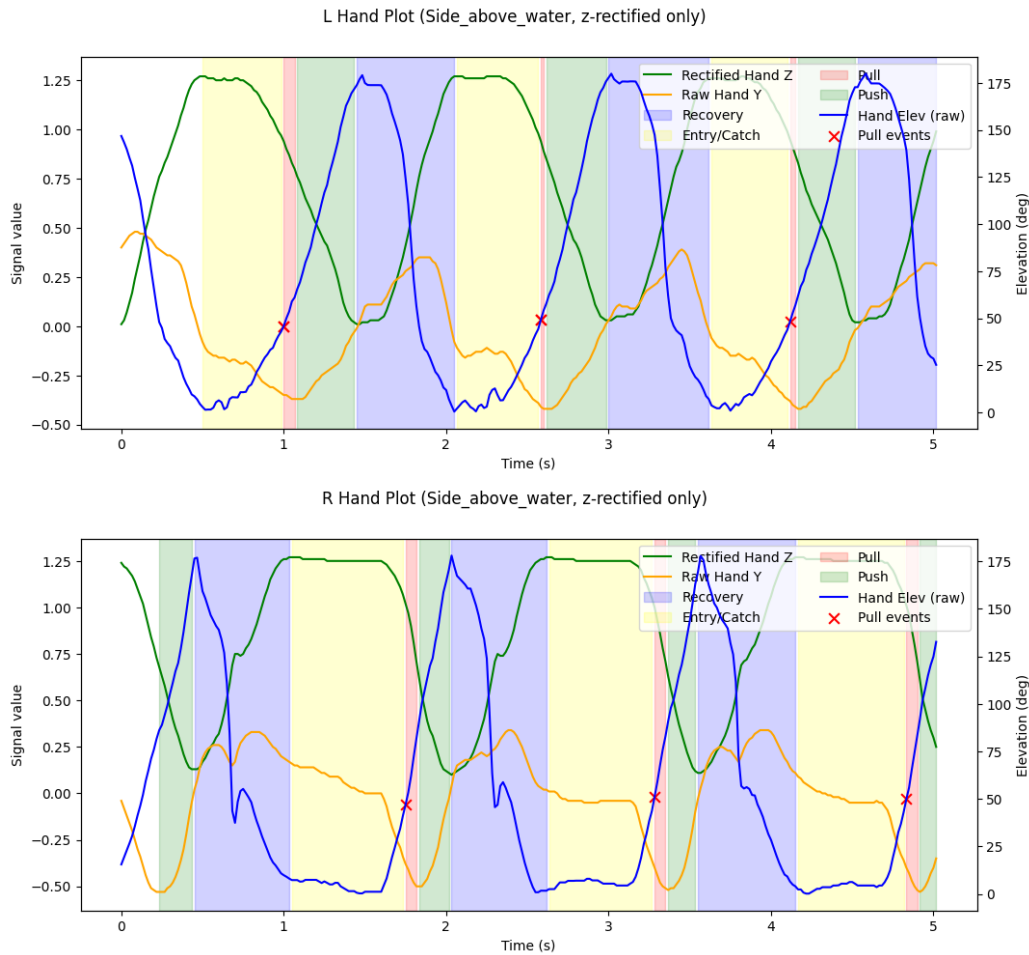


Figure 3.13: The four calculated freestyle swim phase classes overlaid onto the left and right hand positions and shoulder elevation angles for a sample 5 second SwimXYZ video

Once the phase labels were generated from the base data annotations for a given video, they were directly copied into the matching COCO joint annotation view annotation set. This transfer adds the two 'target labels' labeled Left Phase and Right

Phase to each COCO video annotation view set. Subsequently, in later sections, the same phase labels were also transferred to the corresponding video dataset after computer vision models were used to detect and generate swimmer joint positions. Having common phase labels ensures that all datasets are trained against the same target labels, making later comparisons between base, COCO, and CV inputs meaningful.

Chapter 4

Proposed Framework for Freestyle Phase Classification and Stroke Counting

This project uses a full video-to-stroke-count pipeline for freestyle swimming analysis. The goal of the pipeline is to move from raw swimming video to frame-level arm-phase labels, and then from those labels to a final stroke count. The system has five main stages: computer vision pose extraction, pose normalization and windowing, LSTM-based phase classification, HMM/Viterbi temporal decoding, and rule-based stroke counting.

4.1 System Overview

The system begins with a 5-second swimming video as an input. A YOLO pose-estimation model is applied to each frame to estimate swimmer keypoints, including the shoulders, wrists, and hips. These keypoints are then aligned with the corresponding left- and right-arm phase labels produced during dataset generation.

After alignment, each pose sequence is converted into a hip-relative coordinate system, split into fixed-length temporal windows, and passed into a recurrent neural network (LSTM). The LSTM predicts a freestyle phase label for the left and right arms at every frame. Because the LSTM operates on overlapping windows, the window-level predictions are reconstructed into full video-length frame sequences. These frame predictions are then smoothed with a Hidden Markov Model and Viterbi decoder before a final state-machine algorithm counts completed freestyle arm cycles. Figure 4.1 illustrates the complete system flowgraph.

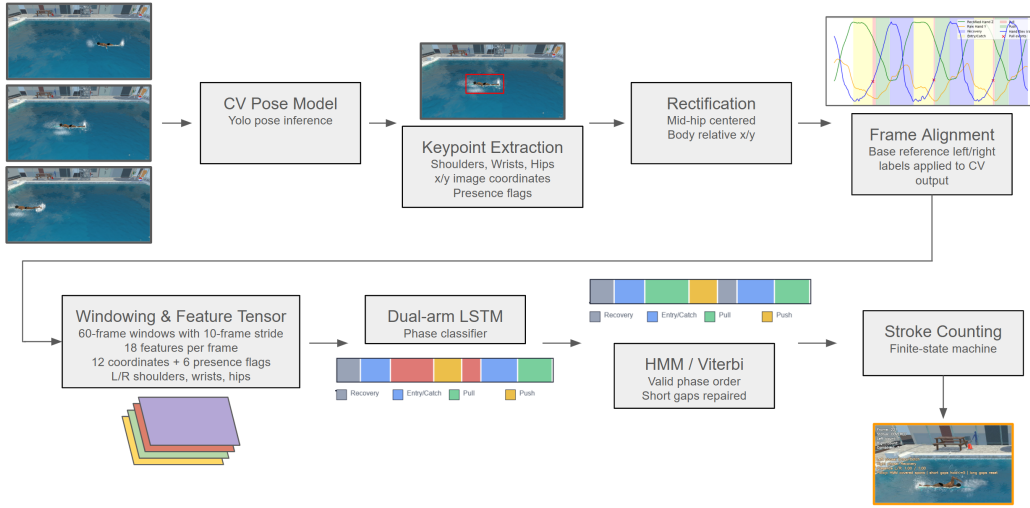


Figure 4.1: Overview of the Automatic Freestyle Stroke Counting Architecture.

4.2 Computer Vision Pose Extraction

Pose estimation was used to convert each video frame into a structured representation of the swimmer's body. In this project, pose estimation refers to predicting two-dimensional (x, y) landmarks on the swimmer's body, such as shoulders, elbows, wrists, and hips, from image frames. A CV model is necessary because freestyle stroke phases are driven by the relative motion of the arms and torso and cannot be determined solely from the image's raw appearance.

Two pose-estimation models were evaluated: YOLO [10] and MediaPipe [11]. YOLO is a family of deep learning models commonly used for object detection and pose estimation. In pose-estimation mode, YOLO predicts human keypoint coordinates and confidence scores for each detected person. MediaPipe Pose Landmarker is a markerless pose-estimation framework that predicts body landmarks along with visibility and presence scores.

The decision to compare YOLO and MediaPipe was directly informed by Caputo et al. (2026) [9], who studied swimmer pose estimation in aquatic environments. Their work used Ultralytics YOLOv8 for main body keypoint detection and Google MediaPipe to support the detection of additional distal landmarks such as hands and feet. Because their study specifically examined swimmer pose estimation under water-related visual challenges, it provided a strong basis for selecting YOLO and MediaPipe as the two computer vision models compared in this project. Replicating the merged YOLO and MediaPipe model outputs into a single set of annotations is too complicated and outside the scope of this research. As such, both models were compared and evaluated on the SwimXYZ video dataset.

Caputo et al. found that aquatic pose estimation is affected by problems that are less common in standard terrestrial pose datasets. These include optical distortion, reflections, bubbles, turbulence, self-occlusion, camera angle, and unusual body orientation. They also showed that model performance varies by body region. Central landmarks such as the hips and shoulders were generally easier to localize, while distal landmarks such as wrists, hands, ankles, and feet were more error-prone. This finding was especially important for the present project because freestyle stroke phase depends strongly on wrist motion. A model that performs well on average but frequently loses the wrists would not be useful for stroke counting.

This research evaluated YOLO and MediaPipe against the Unity-generated COCO

reference keypoints, as these are known ground truths. Tuning the models' hyperparameters maximizes each model's performance on the given SwimXYZ dataset and enables better comparison between them. YOLO26x-pose [10] was swept through its two main hyperparameters: confidence thresholds of 0.05, 0.1, 0.2, 0.3, 0.4, and Intersection over Union (IoU) thresholds of 0.1, 0.3, 0.5, 0.7. The MediaPipe tuning notebook used the heavy Pose Landmarker model, and swept minimum pose detection confidence, pose presence confidence, and tracking confidence over 0.1, 0.3, 0.5, and 0.7.

For each model and hyperparameter setting, several metrics were calculated. The first was the mean joint localization error, $e_{i,t}$, computed as the Euclidean distance between the predicted joint position and the reference joint position:

$$e_{i,t} = \sqrt{(p_{x_{i,t_{pred}}} - p_{x_{i,t_{ref}}})^2 + (p_{y_{i,t_{pred}}} - p_{y_{i,t_{ref}}})^2} \quad (4.1)$$

where $p_{x_{i,t}}$ and $p_{y_{i,t}}$ refer to the pixel x and y coordinates of the joint, i, in a given frame, t. The 'pred' refers to the predicted CV output keypoint while the 'ref' refers to the reference, ground truth, COCO keypoint. The mean error for a joint in all frames was then:

$$MeanError_i = \frac{1}{N_i} \sum_t^{N_i} e_{i,t} \quad (4.2)$$

where, N_i is the number of valid frames used to compute the mean error for a joint i. This error is also reported as ϵ_i , a percentage of image width:

$$\epsilon_i = \frac{MeanError_i}{ImageWidth} * 100 \quad (4.3)$$

Missing-keypoint rate was calculated as the fraction of frames where a joint did not have a valid predicted x/y coordinate:

$$MissingRate_i = \frac{MissingFrames_i}{TotalFrames} \quad (4.4)$$

Temporal jitter was used to measure how much a predicted joint moved from one valid frame to the next:

$$Jitter_i = \frac{1}{N_i - 1} \sum_t \sqrt{(p_{x_{i,t}} - p_{x_{i,t-1}})^2 + (p_{y_{i,t}} - p_{y_{i,t-1}})^2} \quad (4.5)$$

For YOLO, the model confidence score was summarized per joint. For MediaPipe, visibility and presence scores were summarized instead. This distinction matters because YOLO and MediaPipe do not expose exactly the same uncertainty outputs, so each model was evaluated using the confidence-style signals available from its own framework.

The comparison also included a tolerance-based score. For each frame, a body-size-dependent tolerance radius was computed using the reference swimmer pose. A prediction was counted as correct if its distance from the reference keypoint was less than or equal to that joint’s tolerance radius.

$$TP_{i,t} = \begin{cases} 1, & \text{if } e_{i,t} \leq r_{i,t} \\ 0, & \text{otherwise} \end{cases} \quad (4.6)$$

where $e_{i,t}$ is the localization error, $r_{i,t}$ is the tolerance radius, and $TP_{i,t}$ indicates whether the prediction is counted as correct.

The project also calculated a normalized localization accuracy score, similar in spirit to the normalized localization accuracy discussed by Caputo et al. [9] The normalized error was clipped so that very large errors did not dominate the metric:

$$e_{norm_{i,t}} = \min\left(\frac{e_{i,t}}{r_{i,t}}, 1\right) \quad (4.7)$$

The normalized localization score was then:

$$NLA_i = 100 * (1 - \text{mean}(e_{\text{norm}_i,t})) \quad (4.8)$$

A score closer to 100 indicates better localization, while a score closer to 0 indicates poor localization relative to the allowed joint-specific tolerance.

Table 4.1 summarizes the metrics used to evaluate YOLO and MediaPipe. To choose model settings for the downstream LSTM pipeline, a composite score that intentionally weighted wrist quality more heavily than overall body accuracy was used. This was based on the freestyle task itself: the wrists are the most important landmarks for detecting arm phase. The composite score was then calculated using equation 4.9.

Table 4.1: Summary of pose-estimation evaluation metrics.

Metric	Symbol	Purpose
Joint error	$MeanError_i$	Pixel localization error
Error percent	ϵ_i	Error normalized by image width
Missing rate	$MissingRate_i$	Frequency of undetected joints
Temporal jitter	$Jitter_i$	Frame-to-frame keypoint stability
Confidence / visibility	–	Model uncertainty output
Tolerance correctness	$TP_{i,t}$	Whether error is within tolerance
Normalized localization accuracy	NLA_i	Accuracy scaled by tolerance radius
Wrist missing rate	M_{wrist}	Wrist detection reliability
Wrist error	E_{wrist}	Wrist localization quality
Overall missing rate	M_{overall}	Full-body detection reliability
Overall error	E_{overall}	Full-body localization quality
Overall jitter	J_{overall}	Full-body temporal stability
Composite score	–	Final model-selection score

$$\text{Composite Score} = w_1 M_{\text{wrist}} + w_2 E_{\text{wrist}} + w_3 M_{\text{overall}} + w_4 E_{\text{overall}} + w_5 J_{\text{overall}} \quad (4.9)$$

In equation 4.9, M_{wrist} is wrist missing rate, E_{wrist} is wrist localization error as a percent of image width, M_{overall} is the weighted overall missing rate, E_{overall} is the

weighted overall localization error percentage, and $J_{overall}$ is weighted temporal jitter. The constant weight values, w_1 through w_5 , are meant to place an emphasis on wrist joint’s values. For reference, a lower composite score means better pose estimation performance for a given hyperparameter setting. This scoring approach follows the same reasoning as Caputo et al. [9]: pose estimation for swimming should not be judged only by global averages, because errors on small distal joints can matter more than equally sized errors on larger central joints. In fact, the various coefficient values are fixed and derived from Caputo et al. [9]. As expected, the higher weight values are attached to the wrist joint metrics as these are the most important for freestyle detection.

Caputo et al. compared pose models using both standard pose-estimation metrics and swimmer-specific metrics. Standard metrics included mean per-joint position error, percentage of correct keypoints, and object keypoint similarity. Their paper also emphasized a normalized localization accuracy metric, where errors are scaled by an anatomical tolerance radius. This structure allows the metric to distinguish between small localization errors and severe failures, instead of treating every error outside a binary threshold the same way.

4.3 Pose Normalization and Windowing

After the CV pose extraction, the predicted keypoints were aligned with the corresponding COCO-format reference dataset for the same video sequence. This alignment allowed the automatically generated left- and right-arm phase labels from the reference data to be copied onto the CV-derived pose sequence. In this way, the model learned from video-extracted pose inputs while still using the controlled phase labels generated during dataset construction.

The raw keypoint coordinates could not be used directly because they depended

on the swimmer’s position in the frame. A swimmer closer to the top, bottom, left, or right side of the image would produce different coordinate values even if the stroke motion was the same. To reduce this dependence on absolute image position, the pose sequence was normalized around the swimmer’s body. A MidHip point was computed for each frame as the average of the left and right hip coordinates. Each target joint was then converted into a body-relative coordinate by subtracting the MidHip point. This made the feature representation focus more on limb position relative to the swimmer’s torso instead of the swimmer’s location in the video frame.

The x-coordinate was also rectified using its absolute value. This step helped make the representation more consistent across swimming direction. If the swimmer appeared mirrored in the frame, the absolute x-coordinate reduced the chance that the LSTM would learn direction-specific patterns rather than stroke-phase motion patterns.

As the CV model does not always detect the swimmer’s joints, the missing keypoints were filled with zero after normalization. Missing or Nan values cannot be given to LSTM as an input and must be dealt with. However, zero-filled coordinates can be ambiguous because a value of zero could either mean that the joint is close to the MidHip reference point or that the model failed to detect the joint. To avoid hiding this information from the LSTM, a binary presence flag was included for each joint. The flag is equal to 0 if the joint is missing and equal to 1 if detected.

The final LSTM input representation contained 18 features per frame. These included 12 normalized coordinate features from six joints (x,y coordinates for each) and 6 binary presence flags. The retained joints were the left shoulder, right shoulder, left wrist, right wrist, left hip, and right hip. For each joint the x and y coordinates and the presence flag is retained. These joints were selected because they describe the upper-body and arm motion needed for freestyle phase classification while keeping

the feature vector compact. Figure 4.2 shows where each joint corresponds on a swimmer’s body.

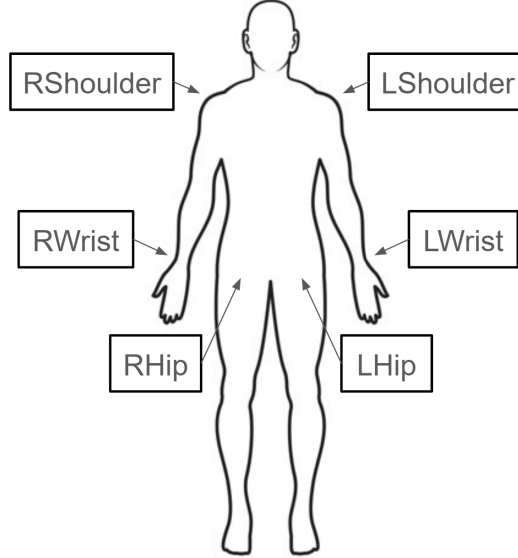


Figure 4.2: Overview of the Automatic Freestyle Stroke Counting Architecture.

Temporal windows were created after the split. Each input window contained 60 consecutive frames, and windows were advanced using a stride of 10 frames. The 60-frame window size was chosen because freestyle phase classification is a temporal problem: a single frame may show where the wrist is, but it does not fully show whether the arm is recovering, entering, pulling, or pushing. The same wrist position can mean different things depending on where the arm came from and where it is moving next. A 60-frame window gives the LSTM enough surrounding context to learn motion patterns rather than relying only on isolated body positions.

The 60-frame length also captures a meaningful portion of a freestyle stroke cycle. At typical video frame rates, 60 frames corresponds to one second of motion. This is long enough to include transitions between multiple stroke phases and often enough to observe a substantial part of an arm cycle. Shorter windows would give the model less temporal context and could make phase boundaries harder to identify. Much

longer windows would increase memory use, reduce the number of training samples, and make training less efficient. Therefore, 60 frames was used as a balance between temporal context and practical model training.

The stride of 10 frames created overlapping windows. This overlap increased the number of training examples while preserving smooth temporal coverage across the original video. It also helped during reconstruction, because each frame could be predicted by multiple nearby windows. These overlapping predictions were later averaged before HMM/Viterbi smoothing, reducing the effect of any single poor window prediction.

Windows were discarded if they contained missing phase labels, invalid numerical values, or no observed target joints across the entire window. Each final input window had shape 60 x 18, where 60 is the number of frames and 18 is the number of pose features per frame. Each target window had shape 60 x 2, where the two target channels corresponded to the left-arm and right-arm phase labels.

4.4 LSTM Phase Classification

After pose normalization and windowing, the next step was to classify the swimmer’s arm phase at each frame. This was treated as a sequence-labeling problem rather than a single-frame classification problem. A single frame can show where the wrist or shoulder is located, but it does not fully describe the direction or timing of the motion. For example, a wrist may appear near the front of the body during both the end of recovery and the start of entry/catch. The phase becomes clearer when the model can see how the pose changes over several frames. For this reason, an LSTM-based recurrent neural network was used.

Long Short-Term Memory networks are designed for sequential data. Unlike a standard feed-forward neural network, an LSTM maintains an internal hidden state as

it processes a sequence. This allows the model to use information from earlier frames when making a prediction at the current frame. This is useful for freestyle swimming because the arm phases follow a repeating temporal order: Recovery, Entry/Catch, Pull, and Push.

The model received one 60-frame window at a time. At a sampling rate of 60 fps, this corresponds to 1 second of video. Furthermore, Each frame contained 18 pose features, consisting of normalized x/y coordinates and joint-presence flags. Therefore, the model input had shape (B, T, F), where B is batch size, T = 60 frames, and F = 18 features per frame. The target output contained one phase label per frame for each arm. Since the model predicted both arms at the same time, the target tensor had shape (B, T, 2), where the final dimension represents the left and right arms. Each label belonged to one of four freestyle phase classes:

- 0 = Recovery
- 1 = Entry/Catch
- 2 = Pull
- 3 = Push

The original labels from the dataset were encoded as 1...4, so they were shifted to 0..3 before training because PyTorch's cross-entropy loss expects class indices beginning at zero.

The model architecture is a dual-arm LSTM. The recurrent encoder consisted of a two-layer LSTM with hidden size 128 and dropout 0.2. The LSTM read the 60-frame pose sequence and produced a hidden representation at every timestep. This allowed the model to make a phase prediction for each frame, rather than producing only one prediction for the entire window. The model layers are summarized in Figure 4.3.

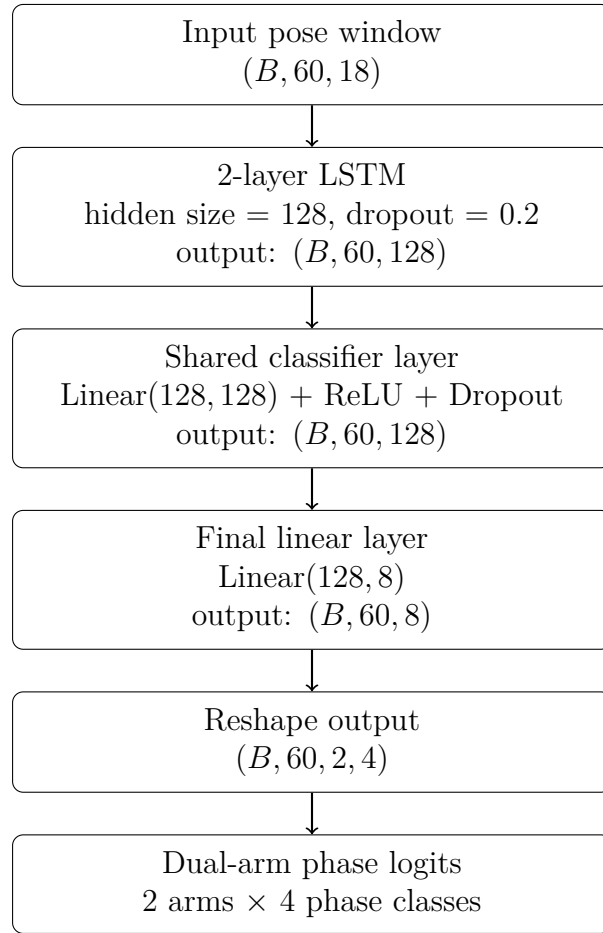


Figure 4.3: Architecture of the dual-arm LSTM model used for frame-level freestyle phase classification. The model receives 60-frame pose windows with 18 pose features per frame and outputs phase logits for the left and right arms at each frame.

The final value of 8 comes from predicting four phase logits for each of two arms, as 2 arms x 4 phase classes = 8 logits per frame.

The same LSTM representation was shared for both arms. This was useful because the left and right arms follow the same phase structure, even though they are offset in time during freestyle. Instead of training two completely separate models, the network learned one temporal representation of swimming motion and then produced separate left-arm and right-arm predictions from that shared representation.

Training used categorical cross-entropy loss [15]. For a single frame and one arm, the loss was:

$$L_{t,a} = -\log\left(\frac{e^{g_{t,a,c}}}{\sum_j e^{g_{t,a,j}}}\right) \quad (4.10)$$

where t is the frame index within the 60-frame input window, a is the arm index, and c is the true phase class for that frame and arm. In this project, a corresponds to either the left arm or the right arm, and c belongs to one of the four freestyle phase classes: Recovery, Entry/Catch, Pull, or Push. Here, j is the phase-class index used in the softmax denominator. It ranges over all four possible phase classes, while c refers specifically to the correct phase class for frame t and arm a . In the equation, $g_{t,a,c}$ is the logit assigned to the correct phase class c , and the denominator sums over the logits for all four possible phase classes.

$$L_{total} = \frac{1}{BTA} \sum_b \sum_t \sum_a L_{b,t,a} \quad (4.11)$$

where b indexes the input window within the batch, t indexes the frame within that window, and a indexes the arm. Therefore, $L_{b,t,a}$ is the cross-entropy loss for one arm at one frame in one training window. Here, B is the batch size, T is the number of frames per window, and $A = 2$ is the number of arms.

The model was trained using the Adam optimizer with learning rate 1e-3 and weight decay 1e-5. A batch size of 128 was used. Gradient clipping with maximum

norm 1.0 was applied during training to prevent unstable updates, which can occur in recurrent models when gradients become too large. A ReduceLROnPlateau scheduler [16] was also used. This scheduler reduced the learning rate when validation loss stopped improving, allowing the model to make smaller updates later in training.

During evaluation, predictions were produced by applying a softmax over the four phase logits and selecting the class with the highest probability:

$$p_{t,a,j} = \frac{e^{g_{t,a,j}}}{\sum_k e^{g_{t,a,k}}} \quad (4.12)$$

where $p_{t,a,j}$ is the predicted probability of phase class j for frame t and arm a , and k indexes over all four possible phase classes in the softmax denominator. The predicted phase is then chosen as the class j with the highest probability, where the predicted $phase_{t,a} = \operatorname{argmax}_j p_{t,a,j}$.

4.5 Hidden Markov Model and Viterbi Temporal Decoding

The LSTM model produced phase predictions on 60-frame overlapping windows. Because each window was classified independently, the same video frame could appear in several different windows and receive several slightly different predictions. Before stroke counting, these window-level predictions had to be reconstructed into one continuous frame-level sequence for each video.

For each test sequence, every window prediction was mapped back to its original absolute frame indices using the stored start and end frame metadata. When multiple windows predicted the same frame, their predicted class probabilities were averaged. This averaging step reduced the effect of any single unstable window prediction and produced a smoother probability sequence over the full video. However, the averaged

LSTM probabilities could still contain frame-to-frame noise. For example, a sequence might briefly jump from Recovery to Pull for one or two frames and then return to Recovery, even though that transition is not biomechanically realistic. To address this, a Hidden Markov Model was used to impose temporal structure on the predictions.

In this project, the hidden states of the HMM were the four freestyle arm phases: Recovery, Entry/Catch, Pull, Push. The LSTM probability vector at each frame was treated as the emission evidence for those hidden states. In other words, the LSTM estimated how likely each phase was from the pose input, and the HMM constrained those estimates based on the expected order of freestyle motion. The HMM consists of three main components:

- Initial state probabilities: $p_{0,i} = P(s_1 = i)$
- Matrix of Transition probabilities: $A_{i,j} = P(s_t = j | s_{t-1} = i)$
- Emission probabilities: $B_{t,j} = P(\text{observation}_t | s_t = j)$

Here, s_t represents the hidden HMM state at frame t . In this project, each hidden state corresponds to one freestyle arm phase: Recovery, Entry/Catch, Pull, or Push. The initial state probabilities describe the likely starting phase, the transition probabilities describe how likely the model is to move from one phase to another, and the emission probabilities describe how well the LSTM output supports each possible hidden phase at a given frame.

The initial state distribution gave the model a starting preference for plausible beginning phases. The following probabilities are not calculated and are based purely on the results in the visual plots produced from the automatic phase labeling logic. The implementation used $p_{0,i} = [0.55, 0.25, 0.10, 0.10]$ which corresponds to [Recovery, Entry/Catch, Pull, Push].

The transition matrix was designed around the expected freestyle phase order: Recovery -> Entry/Catch -> Pull -> Push -> Recovery. The transition matrix is:

$$A_{4,4} = \begin{bmatrix} 0.70 & 0.28 & 0.01 & 0.01 \\ 0.02 & 0.68 & 0.28 & 0.02 \\ 0.02 & 0.02 & 0.68 & 0.28 \\ 0.28 & 0.01 & 0.01 & 0.70 \end{bmatrix} \quad (4.13)$$

Each row represents the current phase, and each column represents the next phase. For example, if the current state is Recovery, the model assigns high probability to staying in Recovery and moderate probability to moving to Entry/Catch. Very low probability is assigned to skipping directly to Pull or Push. This structure reflects the fact that freestyle stroke phases are cyclic and ordered, not arbitrary.

Before decoding, the LSTM emissions were adjusted using joint-presence reliability. The idea was that the model should trust LSTM probabilities more when the relevant joints were visible, and trust them less when pose detection was unreliable. For each frame, arm-specific presence was calculated from the shoulder, wrist, and hip presence flags. The emission vector was then mixed with a uniform distribution:

$$B'_t = r_t B_t + (1 - r_t) U \quad (4.14)$$

where B_t is the original LSTM probability vector, r_t is the reliability score between 0 and 1, and U is a uniform distribution over the four phases (probability = 1/4). When all relevant joints were visible, r_t was close to 1, so the adjusted emission stayed close to the LSTM prediction. When the joints were missing or unreliable, r_t was lower, so the emission moved closer to the uniform distribution. This prevented the HMM from becoming overconfident in frames where the pose estimate was weak.

The most likely phase sequence was then found using Viterbi decoding [17].

Viterbi is a dynamic programming algorithm for finding the single most likely hidden-state path through an HMM. As described in Hui’s [17] explanation of HMMs, the key idea is to replace the summation used in full sequence probability computation with a maximization step, so the algorithm keeps only the best path into each state at each time. After the final frame, the algorithm selects the state with the highest final score and follows the path backward to recover the most likely full phase sequence.

This step converted noisy frame-level LSTM probabilities into a smoother and more biomechanically plausible phase path. Importantly, the HMM did not replace the LSTM. Instead, it acted as a temporal decoder that combined the LSTM’s pose-based evidence with prior knowledge about the expected order of freestyle phases.

4.6 Stroke Counting

After Viterbi decoding, smoothed phase labels for each frame and arm were generated. The final step was to convert this frame-level phase sequence into a stroke count. As the definition of a completed freestyle arm stroke is naturally sequential, a rule-based counting algorithm was used as opposed to another learned model. As previously stated, each arm should pass through the ordered phases of Recovery, Entry/Catch, Pull, and Push. A stroke was counted when an arm completed this full sequence in order. The left and right arms were processed separately because each arm has its own phase timing. The final count was then reported as left-arm strokes, right-arm strokes, and a combined stroke count.

Before counting, the decoded phase sequence was cleaned to reduce the effect of very short, isolated random phase runs. A minimum phase ‘dwell’ threshold of three frames was used. If a predicted phase lasted fewer than three frames, it was treated as likely noise and merged into a neighboring phase label where possible. This prevented one- or two-frame flickers from incorrectly advancing or resetting the stroke-counting

state machine.

The counting algorithm then scanned through each arm’s decoded phase sequence frame by frame. Internally, it tracked which phase was expected next. At the beginning of a potential stroke, the algorithm waited for Recovery. Once Recovery was observed, it advanced to expect Entry/Catch, then Pull, then Push. When Push was completed after the earlier phases had appeared in order, one stroke was counted.

If the sequence returned to Recovery before the full stroke order was completed, the state machine restarted from Recovery. This helped prevent partial or interrupted phase patterns from being counted as full strokes. The implementation also handled short gaps in the decoded sequence. A short gap threshold of five frames was used. Gaps shorter than or equal to this threshold were bridged during counting, while longer gaps were treated as breaks between valid segments. This allowed the algorithm to tolerate brief uncertainty without counting across long, unreliable regions.

This final rule-based step was intentionally kept separate from the LSTM and HMM. The LSTM learned to classify pose windows, the HMM enforced realistic temporal phase order, and the stroke counter applied the final definition of a completed freestyle stroke. Keeping these stages separate made the pipeline easier to interpret and debug, because errors could be traced to pose extraction, phase classification, temporal smoothing, or counting logic.

Chapter 5

Results & Experiments

5.1 LSTM Performance Using Perfect COCO Pose Inputs

The first experiment evaluated the LSTM phase classifier using the perfect (ground-truth) COCO-format pose dataset. This experiment was designed as a best-case test of the temporal model. Because the input keypoints were directly derived from the generated COCO reference data rather than a computer vision model, this setting removed pose-estimation noise from the pipeline. In other words, this experiment asked: if the pose coordinates are accurate, can the LSTM learn to classify freestyle arm phase from keypoint motion alone?

The model was trained for 20 epochs using 60-frame windows as described in the earlier sections. Each input window had a shape of 60×12 , corresponding to the normalized x- and y-coordinates of the selected joints. As previously mentioned, the target output had shape 60×2 , with one phase label for the left arm and one phase label for the right arm at each frame. Across training, validation accuracy improved quickly during the first few epochs and then stabilized, suggesting that

the model learned the main temporal structure of the stroke early and then made smaller refinements later in training. By the final epoch, training accuracy reached 0.9534 and validation accuracy reached 0.9480. On the held-out test set, the model achieved an overall frame-level accuracy of 0.9469, with left-arm accuracy of 0.9360 and right-arm accuracy of 0.9579.

Table 5.1: Summary of COCO LSTM training and test performance.

Split / Epoch	Loss	Overall Acc.	Left Acc.	Right Acc.
Epoch 1 Train	0.3161	0.8929	0.8754	0.9104
Epoch 1 Val	0.2231	0.9252	0.9121	0.9383
Epoch 20 Train	0.1326	0.9534	0.9430	0.9639
Epoch 20 Val	0.1602	0.9480	0.9369	0.9590
Test	0.1600	0.9469	0.9360	0.9579

The small difference between final training accuracy and test accuracy suggests that the model generalized well when the input pose data was clean. This result is important because it shows that the LSTM architecture itself was capable of learning the relationship between pose motion and freestyle phase. Later drops in performance using YOLO or MediaPipe inputs can therefore be interpreted as coming at least partly from pose-estimation error, rather than from the LSTM being unable to learn the phase-classification task.

The confusion matrices in Figure 5.1 show that most errors occurred between neighboring or visually similar phases. Recovery and Entry/Catch were classified with especially high accuracy. Pull was the most difficult phase, with the lowest F1-score in both arms. This is likely because Pull is shorter and more transitional than the other phases, so it contributes fewer frames to the dataset and is more easily confused with adjacent phases, especially Push.

The combined classification report is shown in Table 5.2. Entry/Catch had the highest F1-score at 0.9684, followed by Recovery at 0.9543 and Push at 0.9271. Pull had a lower F1-score of 0.7984. Even in the perfect-pose setting, Pull remained the

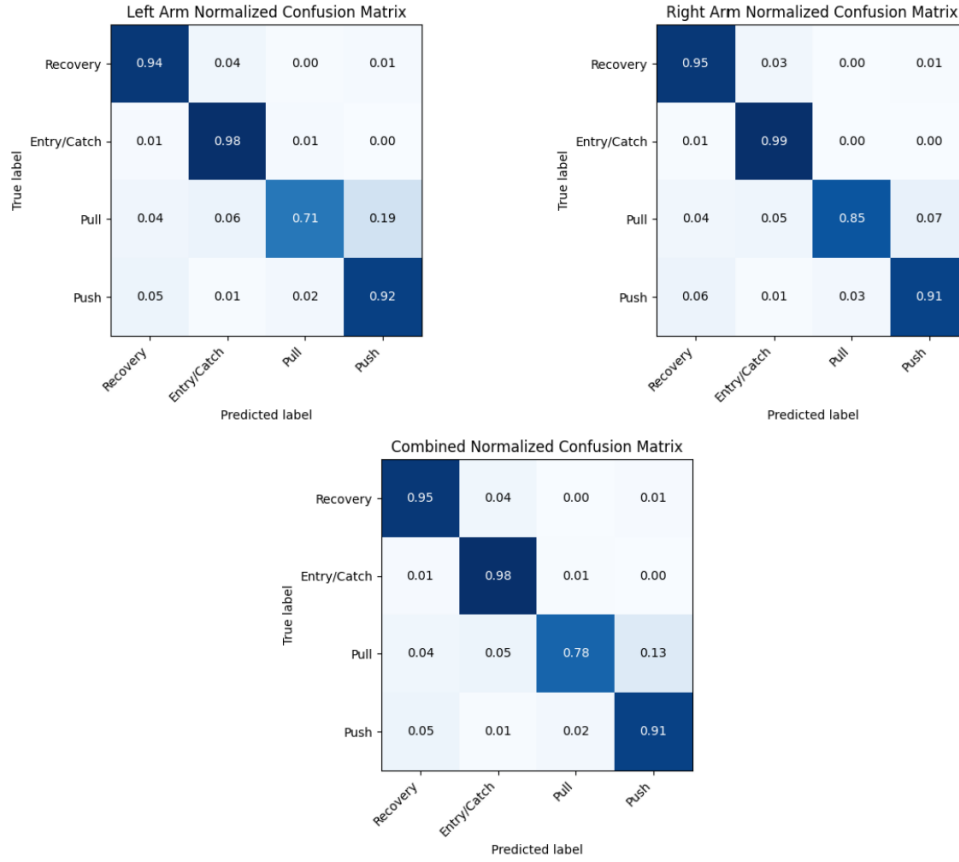


Figure 5.1: Normalized confusion matrices for the COCO LSTM model. Values represent the proportion of each true class assigned to each predicted class.

most challenging class, which suggests that some phase ambiguity comes from the temporal labeling task itself and not only from computer vision error.

Performance was also compared across camera views in Table 5.3. The model performed best on the side-water-level, side-above-water, and side-underwater views, with mean accuracies of around 0.951. The front view had the lowest mean accuracy at 0.9386. This pattern is reasonable because side views provide a clearer view of the arm's forward and backward motion during freestyle, while the front view compresses much of that motion into depth, making phase boundaries harder to distinguish from 2D coordinates alone.

The model was then evaluated across swimmer appearance, water condition, lighting, speed, and swimmer position as seen in Tables 5.4 and ???. These breakdowns

Table 5.2: Combined left- and right-arm classification report for the COCO LSTM model.

Class	Precision	Recall	F1-score	Support
Recovery	0.9598	0.9488	0.9543	570248
Entry/Catch	0.9542	0.9830	0.9684	564058
Pull	0.8135	0.7839	0.7984	74782
Push	0.9403	0.9142	0.9271	280832
Accuracy			0.9469	1489920
Macro Avg	0.9169	0.9075	0.9120	1489920
Weighted Avg	0.9466	0.9469	0.9467	1489920

Table 5.3: COCO LSTM test accuracy by camera view.

View	Windows	Left Acc.	Right Acc.	Mean Acc.
Aerial	2484	0.9357	0.9504	0.9431
Front	2483	0.9276	0.9496	0.9386
Side above water	2492	0.9401	0.9617	0.9509
Side underwater	2484	0.9378	0.9636	0.9507
Side water level	2473	0.9388	0.9641	0.9515

help show whether the LSTM was learning a general stroke pattern or only performing well under a narrow set of visual conditions. Across body appearances, mean accuracy ranged from 0.9382 to 0.9523. Across water conditions, mean accuracy ranged from 0.9413 to 0.9518. The lighting, speed, and position results showed similarly modest variation, suggesting that the perfect-pose LSTM was fairly stable across the simulated conditions.

Table 5.4: COCO LSTM test accuracy by swimmer body appearance.

Body Appearance	Windows	Left Acc.	Right Acc.	Mean Acc.
Skin 0.75, Muscle 2	2575	0.9410	0.9635	0.9523
Skin 0.25, Muscle 8	3426	0.9414	0.9628	0.9521
Skin 0.75, Muscle 8	3096	0.9341	0.9582	0.9462
Skin 0.25, Muscle 2	3319	0.9282	0.9482	0.9382

Overall, the perfect COCO experiment showed that the LSTM can successfully learn to classify freestyle phases when given accurate pose coordinates. The model reached high overall accuracy, generalized across camera views and simulated environ-

Table 5.5: COCO LSTM test accuracy by swimming speed and position.

Condition	Windows	Left Acc.	Right Acc.	Mean Acc.
Speed 2	5794	0.9355	0.9625	0.9490
Speed 3	6622	0.9364	0.9539	0.9451
Position 3.75	5880	0.9376	0.9576	0.9476
Position 1.75	6536	0.9346	0.9581	0.9463

mental conditions, and produced especially strong results for Recovery, Entry/Catch, and Push. The main weakness was Pull, which remained the most difficult phase even without computer vision noise. This establishes an important baseline for the later experiments: if performance decreases when YOLO or MediaPipe keypoints are used, that decrease can be compared against this near-ideal pose-input condition.

5.2 MediaPipe and YOLO Pose-Estimation Comparison

After confirming that the LSTM could classify stroke phase using perfect COCO keypoints, the next experiment compared MediaPipe and YOLO as real computer-vision pose extractors. This comparison was necessary because the final pipeline depends on the quality of the pose sequence given to the LSTM. A pose model could have reasonable average performance but still fail for this task if it frequently loses the wrists, produces unstable frame-to-frame motion, or performs poorly in a selected camera view. Hence, MediaPipe and YOLO were compared across their respective hyperparameter settings to determine which pose-estimation approach yielded the most useful keypoints to be fed downstream into the LSTM. As explained in Section 4.2, both CV models were evaluated on the same 40-video tuning subset with matching COCO ground-truth references. MediaPipe was tested across 64 hyperparameter settings, resulting in 2,560 runs, while YOLO was tested across 20 hyperparameter

settings, resulting in 800 runs.

The evaluation metrics, explained in Section 4.2, include mean joint localization error, missing-keypoint rate, temporal jitter, confidence or visibility scores, tolerance-based accuracy, normalized localization accuracy, and a task-weighted composite score. The composite score placed extra weight on wrist quality because wrist motion is central to freestyle phase classification and stroke counting.

The composite score was designed around the downstream stroke-phase task. Since wrist motion is one of the strongest cues for distinguishing Recovery, Entry/-Catch, Pull, and Push, wrist missing rate and wrist localization error were weighted most heavily, as seen in Equation 4.9. The weight values in Equation 4.9 were chosen as follows: $w_1 = 0.35, w_2 = 0.30, w_3 = 0.20, w_4 = 0.10, w_5 = 0.05$.

The overall best MediaPipe and YOLO configurations are compared in Table 5.6. At the global hyperparameter level, YOLO had the lower best composite score. The best MediaPipe setting used minimum pose detection confidence 0.1, minimum pose presence confidence 0.1, and minimum tracking confidence 0.3. The best YOLO setting used confidence threshold 0.10 and IoU threshold 0.10. YOLO had lower mean joint error, lower wrist error, and lower jitter. MediaPipe had a lower missing-keypoint rate in its best global setting, but its predicted points were less accurately localized.

Table 5.6: Best global MediaPipe and YOLO pose-estimation settings. Lower composite score indicates better performance for the downstream stroke-phase task.

Model	Best Parameters	Mean Error	Error %	Missing Rate	Wrist Error	Wrist Missing	Composite
MediaPipe	det=0.1, pres=0.1, track=0.3	108.38	5.64	0.3009	162.16	0.3009	5.1191
YOLO	conf=0.10, iou=0.10	97.08	5.06	0.4279	142.78	0.4279	4.7913

These view-level results were then used to determine which video view to carry forward. In the previous Section 5.1, all views were used to train the LSTM model. This was a benchmark to see the viability of the LSTM architecture for phase detection. For the final pipeline, only the best set of views would be considered, not

all of them. Not including all camera angles is important because different angles reveal different aspects of the swimmer’s motion. A front view can hide forward-backward arm movement in depth, underwater views can be affected by distortion and occlusion, and aerial views may not show the same arm-phase cues as clearly as side views. Therefore, finding the optimal view or views that maximize the YOLO model performance is crucial.

The strongest result for the final pipeline came from YOLO on the side-above-water view as seen in Table 5.7. In that view, YOLO achieved a composite score of 2.3766, compared with MediaPipe’s 3.1007. YOLO also had lower mean joint error and lower wrist error for this view. This result suggested that side-above-water YOLO provided the best balance of visibility, localization accuracy, and wrist tracking for the freestyle phase-classification task.

Table 5.7: Best MediaPipe and YOLO performance by camera view.

View	Model	Mean Error	Missing Rate	Wrist Error	Composite
Aerial	MediaPipe	68.88	0.2612	94.76	2.7292
Aerial	YOLO	65.69	0.2903	104.81	3.7990
Front	MediaPipe	199.19	0.5257	311.58	7.9379
Front	YOLO	176.61	0.4136	267.30	7.2491
Side above water	MediaPipe	67.17	0.0806	91.02	3.1007
Side above water	YOLO	52.79	0.3459	69.51	2.3766
Side underwater	MediaPipe	121.74	0.9643	148.95	6.9977
Side underwater	YOLO	105.20	0.7052	152.75	5.2115
Side water level	MediaPipe	73.59	0.1171	110.33	2.7657
Side water level	YOLO	78.33	0.5548	107.16	3.9391

Although MediaPipe performed better in some views, especially aerial and side-water-level, YOLO performed best in the view that was most useful for the freestyle stroke-phase task. In the SwimXYZ simulated environment, the side-above-water view always gives a clear lateral view of the arm recovery and pull motion. This also avoids some of the distortion and landmark loss seen in the simulated underwater footage. In the side-above-water view results seen in Table 5.8, YOLO reduced the

left wrist mean error from 105.88 pixels with MediaPipe to 83.34 pixels, and reduced the right wrist error from 81.62 pixels to 55.75 pixels. Based on these results, the final dataset used to train the LSTM model was built using YOLO pose extraction from only side-above-water views. YOLO as opposed to Media was selected because it had the best composite score, the strongest side-above-water performance, and substantially better wrist localization in the view that best represented freestyle arm motion.

Table 5.8: Side-above-water wrist localization error for MediaPipe and YOLO.

Model	Left Wrist Error	Right Wrist Error	Mean Wrist Error
MediaPipe	105.88	81.62	91.02
YOLO	83.34	55.75	69.51

5.3 YOLO Side-Above-Water Dataset and LSTM Results

5.3.1 Developing the Side-Above-Water View Dataset

After the MediaPipe and YOLO comparison, the final dataset was built using the optimized YOLO hyperparameter configuration on the side-above-water view. As mentioned in Section 4.3, this step extracted the various six target joint poses and added binary presence masks for missing joints.

The Figures 5.3 and 5.4 show an example of this process for the left and right wrists. The raw YOLO wrist coordinates are first shown in image space, then transformed into MidHip-normalized coordinates, and finally rectified so that mirrored swimming directions are represented more consistently. The presence mask in the bottom panel shows where YOLO successfully detected the wrist. These plots are useful because they show both the strength and limitation of the real-CV dataset:

YOLO captures the general arm-motion pattern, but the extracted signal contains missing segments and noisier wrist trajectories than the perfect COCO input.

It is important to note that, when developing the dataset, the YOLO model may not detect the swimmer’s pose for extended periods. Such examples, as seen in Figure 5.2, include when the swimmer is out of frame or when reflections or bubbles occlude the swimmer. As mentioned, this was attempted to be overcome by introducing the presence mask into the data. However, extended periods with a presence mask equal to 0, where no detection is made, can rarely result in full 60-frame windows without any YOLO output. Essentially, there may be windows without any actual data inside them. This means that if such data is fed to the LSTM model, the target phase variable, copied from the perfect COCO data, would be attached to an input variable that would be stuck at 0 with a presence mask of 0 as well. Therefore, this data would cause the model to incorrectly learn swimming dynamics. For this reason, during final dataset preparation, 60-frame windows without detections are removed. As long as one frame out of 60 contains a detection, then it is kept in the training, validation, and testing data for the LSTM.

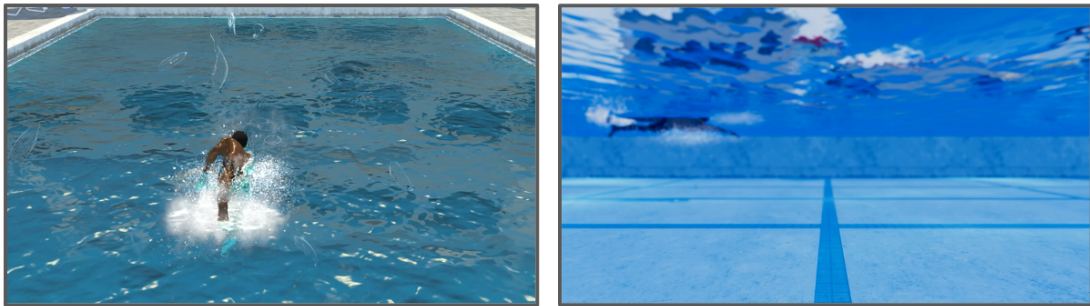


Figure 5.2: Example video frames showing the harsh aquatic environment, including reflected sunlight and bubbles, making CV inference difficult.

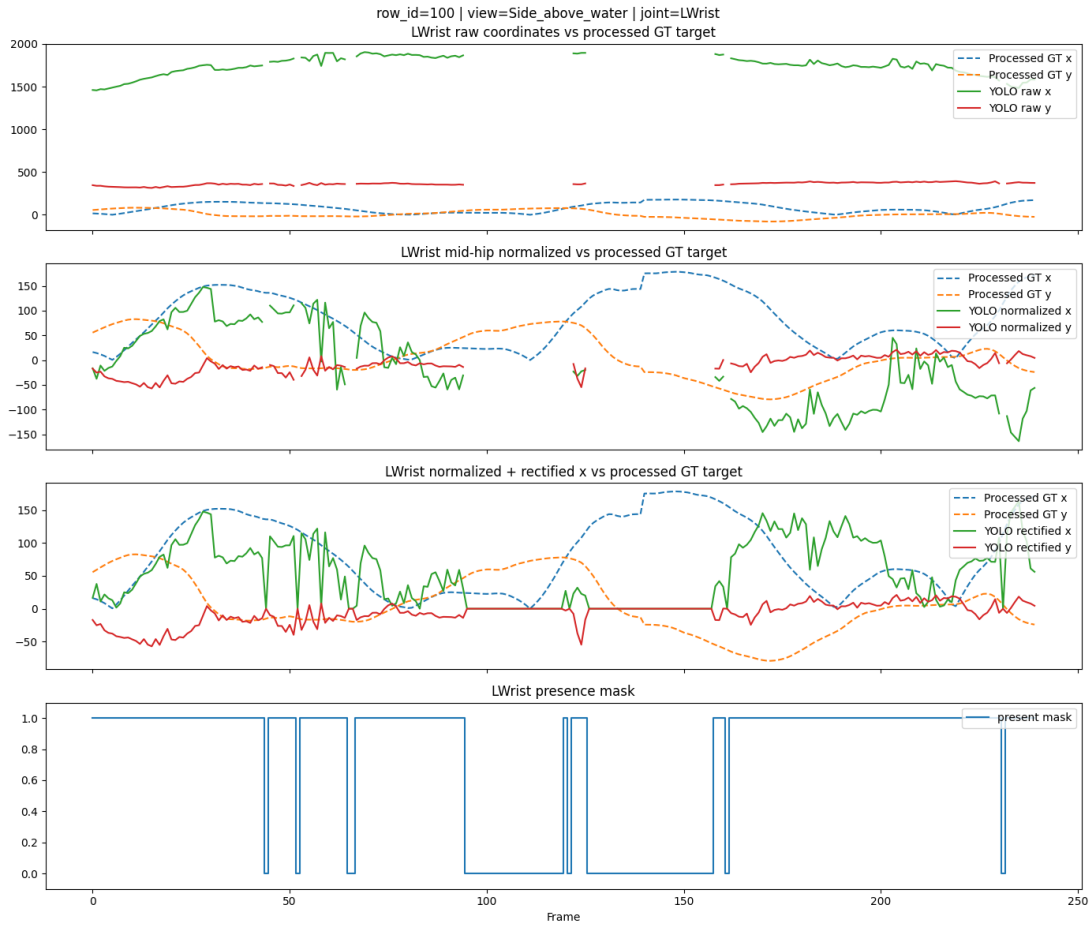


Figure 5.3: YOLO rectification example for the left wrist in the side-above-water view. The plot shows raw image coordinates, MidHip-normalized coordinates, rectified coordinates, and the wrist presence mask.

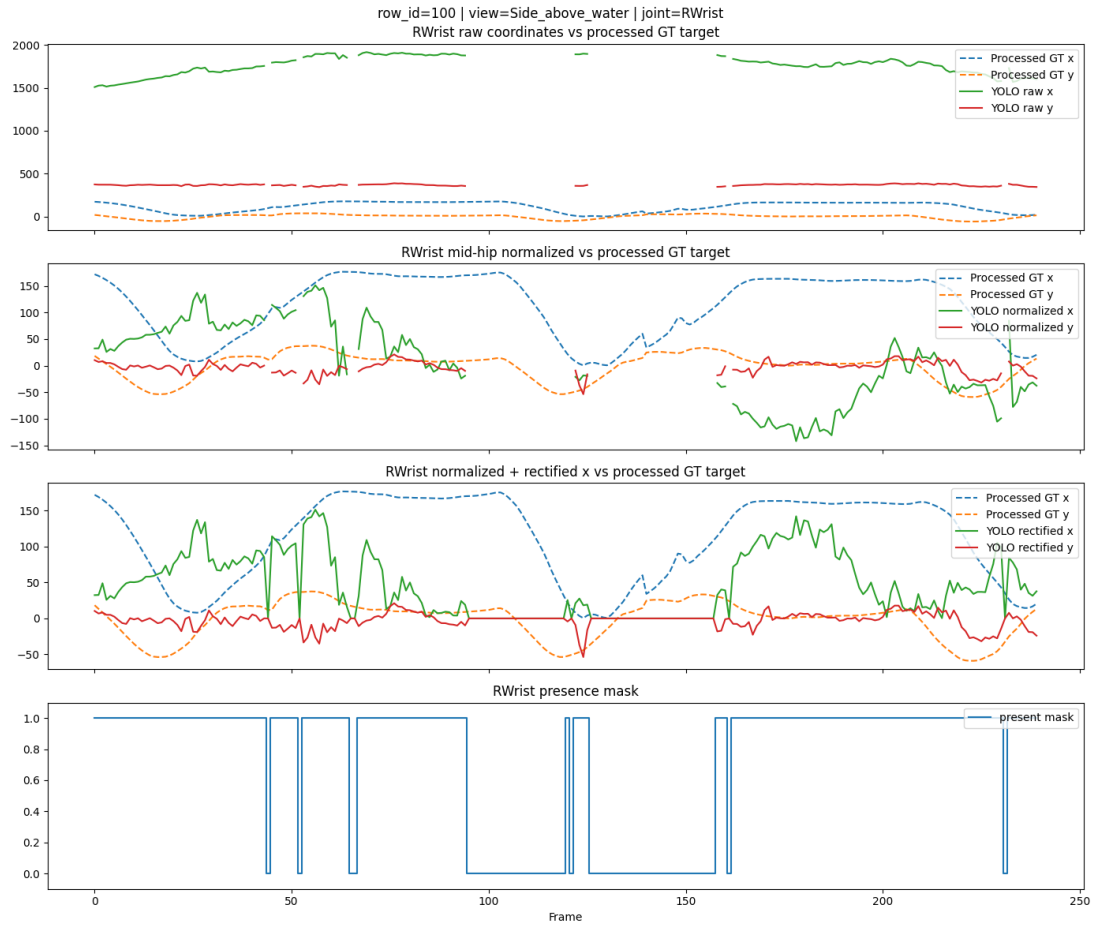


Figure 5.4: YOLO rectification example for the right wrist in the side-above-water view. The presence mask shows frames where YOLO detected or missed the wrist keypoint.

5.3.2 Side-Above-Water View LSTM Model Results

The LSTM was then trained and evaluated using the side-above-water YOLO-derived pose dataset. Compared with the perfect COCO experiment, this was a more realistic test because the model no longer received clean ground-truth pose coordinates. Instead, it received noisy, occasionally missing keypoints from an actual pose-estimation model. On the test set, as seen in Table 5.9, the LSTM model achieved a combined frame-level accuracy of 0.8363. Furthermore, the left-arm accuracy was 0.8211, while the right-arm accuracy was 0.8516.

Table 5.9: YOLO side-above-water LSTM test performance.

Input Dataset	Overall Acc.	Left Arm Acc.	Right Arm Acc.
YOLO side-above-water	0.8363	0.8211	0.8516

This result was lower than the perfect COCO baseline, but it still shows that the temporal model learned to meaningfully predict a swimmer’s freestyle phase from real computer-vision keypoints. The decrease in accuracy is expected because YOLO introduces error, jitter, and missing wrist annotations. These errors are especially important for freestyle phase classification because phase boundaries depend heavily on wrist position and motion.

The combined classification report is shown in Table 5.10. Recovery and Entry/-Catch remained the strongest classes, with F1-scores of 0.8585 and 0.8675. Push also performed reasonably well, with an F1-score of 0.7932. Pull was again the most difficult class, with an F1-score of 0.5442. This mirrors the same trend in the COCO experiment, where Pull was also the weakest class. However, as real YOLO keypoints were used, the results are correspondingly lower.

The normalized confusion matrices in Figure 5.5 show where the model made most of its errors. Recovery and Entry/Catch were usually classified correctly while Pull was the main failure point. In the combined matrix, only 0.48 of Pull frames were

Table 5.10: Combined classification report for the YOLO side-above-water LSTM model.

Class	Precision	Recall	F1-score	Support
Recovery	0.8535	0.8636	0.8585	104237
Entry/Catch	0.8467	0.8894	0.8675	105271
Pull	0.6202	0.4848	0.5442	13514
Push	0.8225	0.7659	0.7932	52138
Accuracy			0.8363	275160
Macro Avg	0.7857	0.7509	0.7659	275160
Weighted Avg	0.8336	0.8363	0.8341	275160

classified as Pull, while 0.26 were classified as Entry/Catch and 0.19 as Push. This makes sense as the Pull phase is the shortest during the stroke cycle, suggesting it is an underrepresented class in the YOLO feature space. Furthermore, since Pull occurs between Entry/Catch and Push, noisy wrist localization can make its boundaries difficult to identify. This can all lead to the poor training and test results of the Pull class.

Performance also varied across swimmer and environmental conditions. The model performed best for 'Swimmer Skin 0,75 Muscle 8', with a mean accuracy of 0.8568, and lowest for Swimmer Skin 0,25 Muscle 2, with a mean accuracy of 0.8052. Water and lighting conditions also affected performance, with mean accuracy ranging from 0.8182 to 0.8511 across water settings and from 0.7999 to 0.8592 across lighting settings. Position had one of the clearest effects: position 1,75 achieved 0.8797 mean accuracy, while position 3,75 achieved 0.7855. For reference, position is a hyperparameter setting developed by the SwimXYZ researchers to denote different fixed distances between the swimmer/pool and camera view. A smaller position value means a closer camera view and similarly a higher position values refers to farther camera view. The results are seen in Table 5.11.

Overall, the YOLO side-above-water experiment shows that the full CV to LSTM pipeline can classify freestyle stroke phases, but with a clear performance drop com-

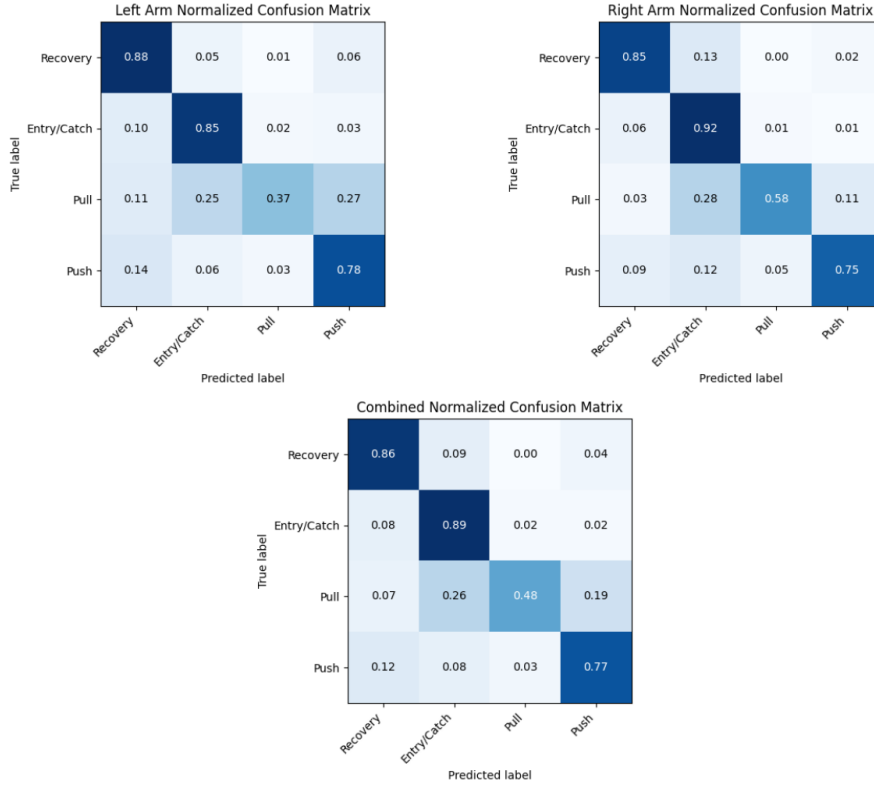


Figure 5.5: Normalized confusion matrices for the YOLO side-above-water LSTM model. Pull was the most difficult phase, with many Pull frames confused with Entry/Catch or Push.

Table 5.11: YOLO side-above-water LSTM accuracy by swimmer position.

Position	Windows	Left Acc.	Right Acc.	Mean Acc.
position_1,75	1238	0.8671	0.8923	0.8797
position_3,75	1055	0.7672	0.8038	0.7855

pared with the perfect COCO baseline. This gap is important because it isolates the practical cost of moving from ideal pose inputs to real pose extraction. The LSTM still performed well on the main phase classes, especially Recovery and Entry/Catch, but Pull remained difficult and became more sensitive to YOLO noise. These results support the later use of HMM/Viterbi smoothing and rule-based stroke counting, since the raw frame-level predictions contain realistic errors that need to be corrected before reliable stroke counts can be produced.

5.3.3 Manual Stroke Count Validation

After the windows were passed through the LSTM model, they were then fed to the HMM/Viterbi decoder and the rule-based stroke counter. To validate the results of this research’s counting method, ground-truth values had to be developed. The SwimXYZ dataset consists solely of video data and does not include phase or stroke counts. Therefore, the automatic stroke-counting system was compared with manually generated counts. This step was necessary because the frame-level phase accuracy shown in the previous section does not directly measure the pipeline’s final goal: counting completed freestyle strokes. Furthermore, a model can correctly classify most frames but still misses a transition that changes the final count. Similarly, a few frame-level errors may not matter if the overall phase sequence still produces the correct completed stroke cycle.

Manual validation was completed on 20 selected videos from the LSTM test set. Unlike randomly selecting 20 videos, these videos were selected based on their meta-data and results after passing through the LSTM and stroke-counting pipeline. Specifically, a 20-video subset was developed from different combinations of these conditions: high-coverage and lower-coverage clips, clips with fewer and more predicted strokes, clips where the predicted stroke count between the two arms agreed, and clips where

the stroke count of the two arms disagreed.

This was done so that the manual validation set would include both easier and harder High-coverage clips with balanced left/right counts, representing cleaner cases, while lower-coverage clips or clips with left/right count disagreement are more likely to be edge cases that show the pipeline failing. Overall, this helped ensure that the validation set included a range of easier and harder examples rather than only high-confidence cases.

Subsequently, manual annotation was done on the selected clips. This was done blind, without any indication of the automatically generated stroke count given by the pipeline, to prevent the user from subconsciously being biased by the model output. The manual counts were entered separately for left-arm strokes, right-arm strokes, and total strokes. These manual counts were then merged with the automatic stroke-count outputs to compute signed error, absolute error, percent error, exact total-count accuracy, and within-one total-count accuracy.

Table 5.12: Summary of manual stroke count validation results.

Metric	Value
Number of manually reviewed videos	20
Left-arm mean absolute error	0.60 strokes
Right-arm mean absolute error	0.70 strokes
Total mean absolute error	0.80 strokes
Total mean percent error	23.25%
Exact total-count accuracy	35.00%
Within-1 total-count accuracy	85.00%

The summary results are shown in Table 5.12. Across the 20 manually reviewed clips, the automatic pipeline had a total mean absolute error of 0.80 strokes. The left-arm mean absolute error was 0.60 strokes, while the right-arm mean absolute error was slightly higher at 0.70 strokes. The total mean percent error was 23.25%. However, this percentage should be interpreted with caution, as the SwimXYZ dataset contains only 5-second standalone videos, meaning validation clips were short and

often contained only a few total strokes. In a four-stroke clip, for example, a one-stroke error already corresponds to a 25% error.

The exact total-count accuracy was 35.00%, meaning that the automatic pipeline exactly matched the manual total stroke count in 7 of the 20 clips. However, the within-one total-count accuracy was much higher at 85.00%, meaning that 17 of the 20 clips were either exactly correct or off by only one total stroke. This suggests that the HMM/Viterbi and rule-based counting systems were usually close to the manual count, even though they were not always exact.

Table 5.13: Manual validation results comparing manual stroke counts with automatic HMM/Viterbi stroke counts.

Video ID	Manual L	Manual R	L Abs. Error	R Abs. Error	Combined Stroke Count	Total Error	Total Percent Error
210	2	2	1	1	2	-2	50.00%
10	1	1	1	0	1	-1	50.00%
557	1	2	0	1	2	-1	33.33%
306	3	1	3	1	2	-2	50.00%
235	2	2	0	0	4	0	0.00%
80	3	1	1	2	5	1	25.00%
85	2	2	0	1	5	1	25.00%
192	2	2	0	1	5	1	25.00%
270	2	2	0	1	5	1	25.00%
286	2	2	0	1	5	1	25.00%
547	2	2	0	1	5	1	25.00%
558	2	3	0	0	5	0	0.00%
149	0	0	0	0	0	0	N/A
263	1	0	1	1	1	0	0.00%
561	1	1	0	0	2	0	0.00%
516	2	2	0	0	4	0	0.00%
494	3	1	2	2	4	0	0.00%
64	2	1	1	0	2	-1	33.33%
146	2	2	2	0	2	-2	50.00%
396	2	2	0	1	3	-1	25.00%

The per-video results in Table 5.13 show that most of the errors were small in absolute count. Out of the 20 manually reviewed clips, 7 clips had an exact total stroke count, 10 clips were off by one total stroke, and only 3 clips were off by two total strokes. No clip had a total absolute error greater than two strokes. This pattern is important because it shows that the automatic system rarely produced a count that was far from the manual count, even when it did not exactly match.

The signed error values also suggest that the model was not strongly biased toward over-counting. Across all 20 clips, the manual total was 68 strokes, while the

automatic total was 64 strokes. This corresponds to an average signed total error of -0.20 strokes per clip, meaning the model slightly undercounted overall. Some clips were over-counted by one stroke, especially where the automatic right-arm count was higher than the manual count. Other clips were undercounted, such as video 210, 306, and 146, where the automatic count was two strokes lower than the manual count.

Overall, the manual validation results show that the complete pipeline produced stroke counts that were usually close to the manually counted ground truth. The exact count accuracy was limited, but the within-one accuracy of 85.00% is encouraging given the short clip lengths and the noisy real-CV pose inputs. The remaining errors likely come from missed or extra phase transitions after HMM/Viterbi decoding, especially when YOLO wrist detections are noisy or when one arm’s phase sequence is less reliable than the other. These results support the use of HMM/Viterbi smoothing and rule-based counting as a practical pipeline for stroke counting in swimming. It also shows that future improvements should focus on reducing missed transitions and improving arm-specific count consistency.

Chapter 6

Conclusion

This thesis developed a video-based pipeline for classifying freestyle phases and counting freestyle strokes in competitive swimming. The central tenet of this pipeline was to treat stroke counting as more than simply finding repeated peaks or isolated events. Instead, freestyle strokes were modeled as a sequence of arm phases: Recovery, Entry/Catch, Pull, and Push. By first predicting these phases and then counting completed phase cycles, each stroke can be counted as a completed phase sequence rather than an isolated motion peak. This makes the stroke count more robust and interpretable.

The results show that this approach is promising. When the LSTM model was trained and tested using perfect COCO pose keypoints, it achieved an overall test accuracy of 0.9469. This result shows that when the pose data is clean, the model can learn the relationship between joint motion and freestyle stroke phase. Recovery, Entry/Catch, and Push were especially well classified. Pull was the weakest class, even in this ideal setting, suggesting it is a more difficult phase to identify because it is the shortest temporally and occurs between two longer neighboring phases.

The comparison between MediaPipe and YOLO also showed how important pose-estimation quality is for the full system. YOLO performed best for the side-above-

water view, especially because it localized the wrists more accurately in that view. Since wrist motion is one of the most important cues for freestyle phase detection, the side-above-water view was used for the final pipeline. With these keypoints, the LSTM reached an overall test accuracy of 0.8363, which, as expected, was lower than the perfect COCO baseline. Although this result cannot be directly compared with manual frame-level phase labels because such labels were not available, it shows that the model retained much of the temporal phase information despite missing, jittery, and imperfect pose estimates. The practical usefulness of these predictions was instead evaluated through manual stroke-count validation.

The final stroke-count validation also showed encouraging results. Across 20 manually reviewed clips, the pipeline had a total mean absolute error of 0.80 strokes. It matched the exact manual total count in 35.00% of the clips and was within one stroke in 85.00% of the clips. Because each SwimXYZ video is only 5 seconds long and contains around 4 to 5 strokes on average, even a one-stroke error can look large as a percentage. For that reason, the within-one result is especially useful: it shows that the system was usually close to the manual count, even when it was not perfectly correct.

Despite this, several limitations remain. The phase labels were generated automatically from synthetic 3D pose data rather than being manually annotated by experts. This made it possible to create labels at scale, but it also means that some phase boundaries may not be perfectly accurate. The work also used synthetic videos. Therefore, these results do not show how the system would perform on real pool footage with real swimmers, camera motion, and more unpredictable water conditions. Furthermore, this research focused solely on the side-above-water view, even though real-world systems may operate across various camera angles. Finally, the manual validation set was small, so a larger set of reviewed videos would be needed

to better understand the reliability of the stroke counts.

Future work could improve the pipeline in several ways. The Pull phase should be studied more closely, as it was the most difficult to detect accurately in both the perfect-pose and YOLO-based experiments. Better phase-labeling rules, class balancing, or phase-duration constraints could help improve this result. The system could also be tested on real swimming videos to measure how well the synthetic training data transfers to real environments. Fine-tuning the pose or temporal model on a small amount of real, annotated data is likely to improve performance. Finally, using multiple camera views or stronger tracking methods could help reduce missing wrist detections and produce more consistent left- and right-arm stroke counts.

Overall, this thesis shows that synthetic swimming data can support a full pipeline for freestyle phase classification and stroke counting. The system performance demonstrates that pose-based temporal modeling can turn raw swimming video into useful, interpretable stroke information. By counting strokes across completed phase sequences, the pipeline links the final count to the swimmer’s actual movement, an important step toward more scalable, explainable swimming analytics.

Bibliography

- [1] C. Didier, S. Chaliès, and J.-C. Chatard, “A new index of coordination for the crawl: Description and usefulness,” *International journal of sports medicine*, vol. 21, pp. 54–9, 02 2000.
- [2] Q. Wang, L. De Baets, A. Timmermans, W. Chen, L. Giacolini, T. Matheve, and P. Markopoulos, “Motor control training for the shoulder with smart garments,” 07 2017.
- [3] B. Victor, Z. He, S. Morgan, and D. Miniutti, “Continuous video to simple signals for swimming stroke detection with convolutional neural networks,” in *2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2017.
- [4] T. Woinoski and I. V. Bajić, “Swimmer stroke rate estimation from overhead race video,” in *2021 IEEE International Conference on Multimedia & Expo Workshops (ICMEW)*, 2021.
- [5] T. Tran, H. A. Joseph, K. Lee, K. T. W. Choo, D. Ma, S. Foong, T. Kandappu, J. Ko, and R. Balan, “Dronaquatics: Real-time swimming analytics using drone captured imagery,” in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2026.

- [6] M. Cortesi, A. Giovanardi, G. Gatta, A. L. Mangia, S. Bartolomei, and S. Fantozzi, “Inertial sensors in swimming: Detection of stroke phases through 3d wrist trajectory,” *Journal of Sports Science and Medicine*, vol. 18, pp. 438–447, 2019. [Online]. Available: <http://www.jssm.org>
- [7] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, “Microsoft coco: Common objects in context,” 2015. [Online]. Available: <https://arxiv.org/abs/1405.0312>
- [8] R. P. Dubois, D. V. Thiel, and D. A. James, “Using image processing for biomechanics measures in swimming,” *Procedia Engineering*, vol. 34, pp. 807–812, 2012.
- [9] A. Caputo, A. Scocco, and P. Castellini, “Problem-related performance metrics of deep learning models: Application to swimmer pose estimation in underwater environments,” *Measurement: Digitalization*, vol. 6, p. 100029, 2026.
- [10] G. Jocher and J. Qiu, “Ultralytics yolo26,” 2026. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [11] C. L. et al., “Mediapipe: A framework for building perception pipelines,” 2019. [Online]. Available: <https://arxiv.org/abs/1906.08172>
- [12] G. Fiche, V. Sevestre, C. Gonzalez-Barral, S. Leglaive, and R. Séguier, “Swimxyz: A large-scale dataset of synthetic swimming motions and videos,” in *Proceedings of the 16th ACM SIGGRAPH Conference on Motion, Interaction and Games*, 2023, pp. 1–7.
- [13] P. Li, K. Aberman, Z. Zhang, R. Hanocka, and O. Sorkine-Hornung, “Ganimator: neural motion synthesis from a single sequence,” *ACM Transactions on Graphics*, vol. 41, no. 4, p. 1–12, 2022. [Online]. Available: <http://dx.doi.org/10.1145/3528223.3530157>

- [14] CMU Perceptual Computing Lab, “OpenPose doc: Output,” 2021, OpenPose 1.7.0 documentation. Accessed: 2026-05-11. [Online]. Available: https://cmu-perceptual-computing-lab.github.io/openpose/web/html/doc/md_doc_02_output.html
- [15] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, V. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: <http://neurips.cc>
- [16] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. Luk, B. Maher, Y. Pan, C. Puhersch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews, G. Chanan, P. Wu, and S. Chintala, “Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation,” in *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, 2024. [Online]. Available: <https://docs.pytorch.org/assets/pytorch2-2.pdf>
- [17] J. Hui, “Machine learning — hidden markov model (hmm),” <https://jonathan-hui.medium.com/machine-learning-hidden-markov-model-hmm-31660d217a61>, Aug. 2019, medium. Accessed: 2026-05-07.